let us see how to use this representation for traversing the tree. For example, if we are at node B and want to move to its parent node A, then we just need to perform ⊕ on its left content with its left child address (we can use right child also for going to parent node).

Similarly, if we want to move to its child (say, left child D) then we have to perform ⊕ on its left content with its parent node address. One important point that we need to understand about this representation is: When we are at node B, how do we know the address of its children D? Since the traversal starts at node root node, we can apply ⊕ on root's left content with NULL. As a result we get its left child, B. When we are at B, we can apply ⊕ on its left content with A address.

6.11 Binary Search Trees (BSTs)

Why Binary Search Trees?

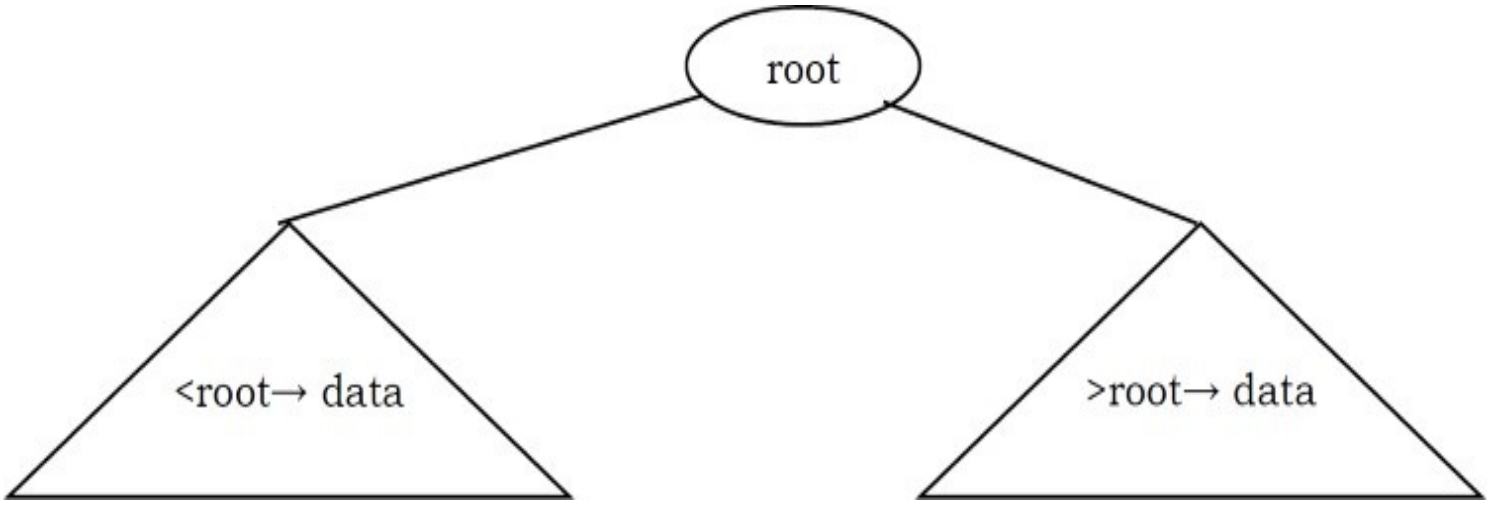
In previous sections we have discussed different tree representations and in all of them we did not impose any restriction on the nodes data. As a result, to search for an element we need to check both in left subtree and in right subtree. Due to this, the worst case complexity of search operation is O(n).

In this section, we will discuss another variant of binary trees: Binary Search Trees (BSTs). As the name suggests, the main use of this representation is for *searching*. In this representation we impose restriction on the kind of data a node can contain. As a result, it reduces the worst case average search operation to O(*logn*).

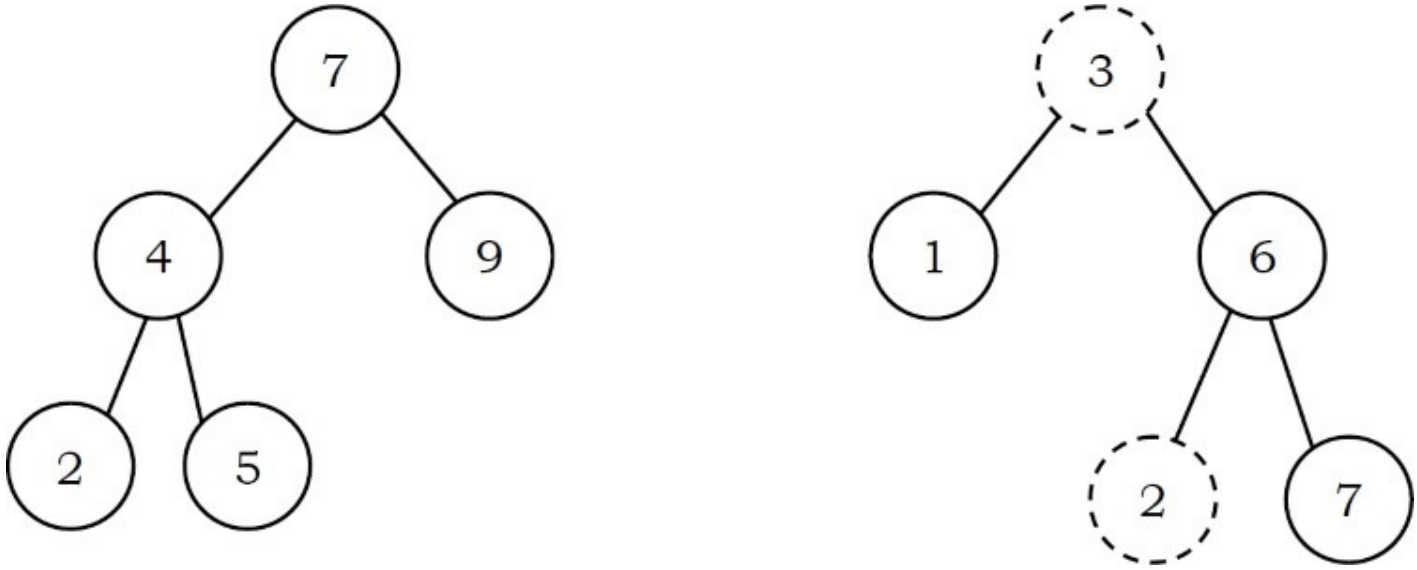
Binary Search Tree Property

In binary search trees, all the left subtree elements should be less than root data and all the right subtree elements should be greater than root data. This is called binary search tree property. Note that, this property should be satisfied at every node in the tree.

- The left subtree of a node contains only nodes with keys less than the nodes key.
- The right subtree of a node contains only nodes with keys greater than the nodes key.
- Both the left and right subtrees must also be binary search trees.



Example: The left tree is a binary search tree and the right tree is not a binary search tree (at node 6 it's not satisfying the binary search tree property).



Binary Search Tree Declaration

There is no difference between regular binary tree declaration and binary search tree declaration. The difference is only in data but not in structure. But for our convenience we change the structure name as:

```
struct BinarySearchTreeNode{
    int data;
    struct BinarySearchTreeNode *left;
    struct BinarySearchTreeNode *right;
};
```

Operations on Binary Search Trees

Main operations: Following are the main operations that are supported by binary search trees:

- Find/ Find Minimum / Find Maximum element in binary search trees
- Inserting an element in binary search trees
- Deleting an element from binary search trees

Auxiliary operations: Checking whether the given tree is a binary search tree or not

- Finding k^{th}-smallest element in tree
- Sorting the elements of binary search tree and many more

Important Notes on Binary Search Trees

- Since root data is always between left subtree data and right subtree data, performing inorder traversal on binary search tree produces a sorted list.
- While solving problems on binary search trees, first we process left subtree, then root data, and finally we process right subtree. This means, depending on the problem, only the intermediate step (processing root data) changes and we do not touch the first and third steps.
- If we are searching for an element and if the left subtree root data is less than the element we want to search, then skip it. The same is the case with the right subtree.. Because of this, binary search trees take less time for searching an element than regular binary trees. In other words, the binary search trees consider either left or right subtrees for searching an element but not both.
- The basic operations that can be performed on binary search tree (BST) are insertion of element, deletion of element, and searching for an element. While performing these operations on BST the height of the tree gets changed each time. Hence there exists variations in time complexities of best case, average case, and worst case.
- The basic operations on a binary search tree take time proportional to the height of the tree. For a complete binary tree with node n, such operations runs in O(lgn) worst-case time. If the tree is a linear chain of n nodes (skew-tree), however, the same operations takes O(n) worst-case time.

Finding an Element in Binary Search Trees

Find operation is straightforward in a BST. Start with the root and keep moving left or right using the BST property. If the data we are searching is same as nodes data then we return current node.

If the data we are searching is less than nodes data then search left subtree of current node; otherwise search right subtree of current node. If the data is not present, we end up in a NULL

link.

```
struct BinarySearchTreeNode *Find(struct BinarySearchTreeNode *root, int data ){
    if( root == NULL )
        return NULL;
    if( data < root→data )
        return Find(root→left, data);
    else if( data > root→data )
        return( Find( root→right, data );
    return root;
}
```

Time Complexity: O(*n*), in worst case (when BST is a skew tree). Space Complexity: O(*n*), for recursive stack.

Non recursive version of the above algorithm can be given as:

```
struct BinarySearchTreeNode *Find(struct BinarySearchTreeNode *root, int data ){
    if( root == NULL )
        return NULL;
    while (root) {
        if(data == root→data)
            return root;
        else if(data > root→data)
            root = root→right;
        else root = root→left;
    }
    return NULL;
}
```

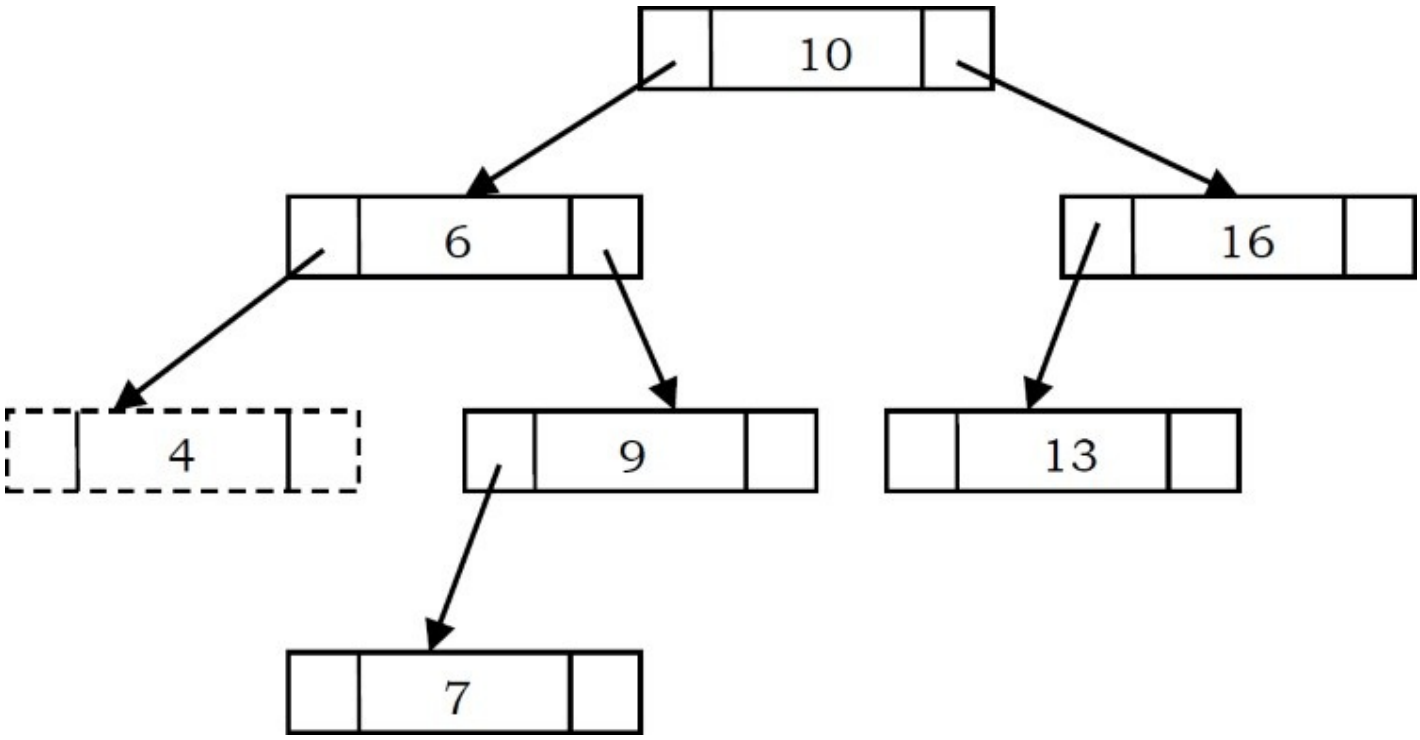
Time Complexity: O(*n*). Space Complexity: O(1).

Finding Minimum Element in Binary Search Trees

In BSTs, the minimum element is the left-most node, which does not has left child. In the BST below, the minimum element is **4**.

```
struct BinarySearchTreeNode *FindMin(struct BinarySearchTreeNode *root){
    if(root == NULL)
        return NULL;
    else  if( root→left == NULL )
        return root;
    else
        return FindMin( root→left );
}
```

Time Complexity: O(*n*), in worst case (when BST is a *left skew* tree).
Space Complexity: O(*n*), for recursive stack.



Non recursive version of the above algorithm can be given as:

```
struct BinarySearchTreeNode *FindMin(struct BinarySearchTreeNode * root ) {
    if( root == NULL )
        return NULL;
    while( root→left != NULL )
        root = root→left;
    return root;
}
```

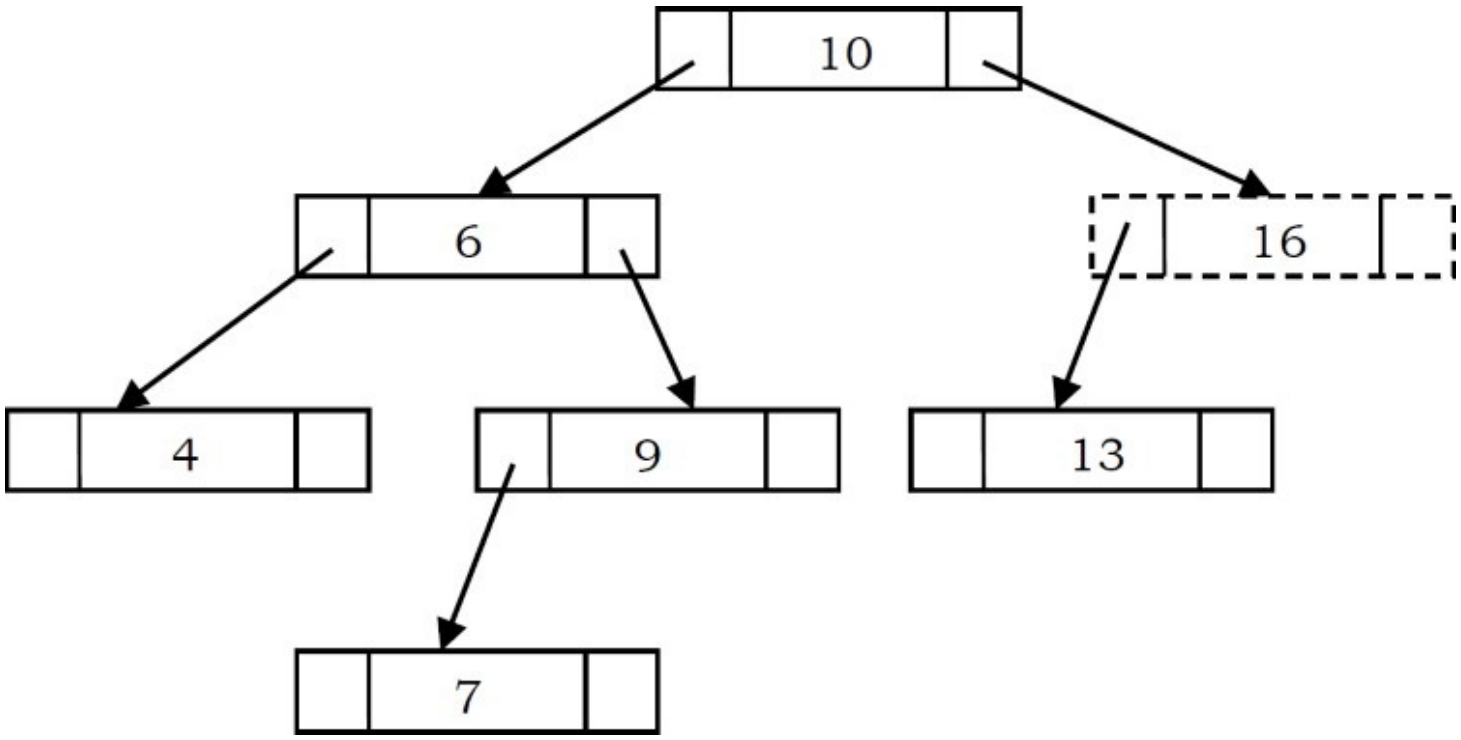
Time Complexity: O(*n*). Space Complexity: O(1).

Finding Maximum Element in Binary Search Trees

In BSTs, the maximum element is the right-most node, which does not have right child. In the BST below, the maximum element is **16**.

```
struct BinarySearchTreeNode *FindMax(struct BinarySearchTreeNode *root) {
    if(root == NULL)
        return NULL;
    else if( root→right == NULL )
        return root;
    else return FindMax( root→right );
}
```

Time Complexity: O(*n*), in worst case (when BST is a *right skew* tree).
Space Complexity: O(*n*), for recursive stack.



Non recursive version of the above algorithm can be given as:

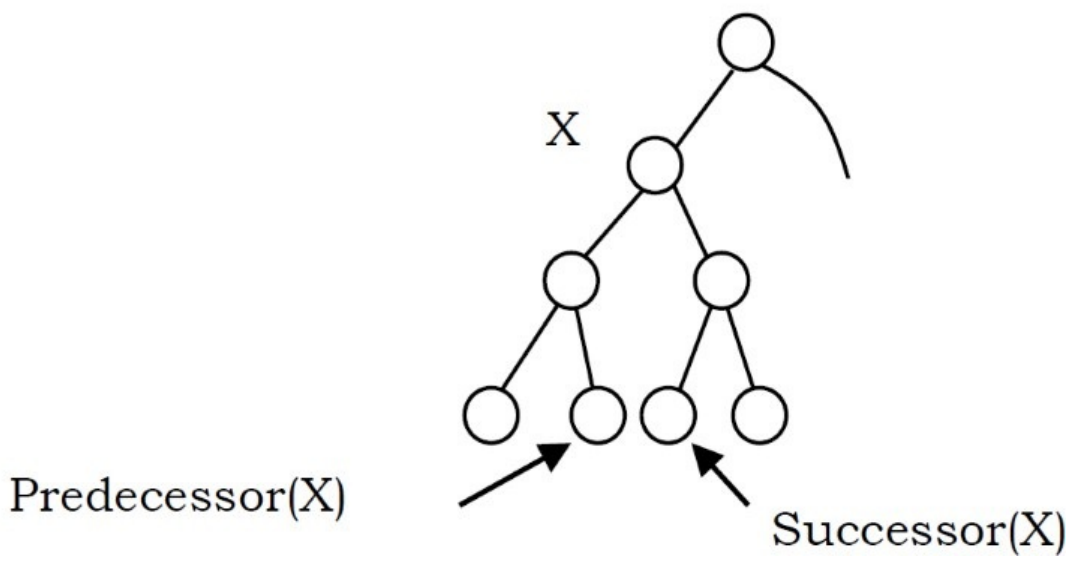
```
struct BinarySearchTreeNode *FindMax(struct BinarySearchTreeNode * root ) {
    if( root == NULL )
        return NULL;
    while( root→right != NULL )
        root = root→right;
    return root;
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

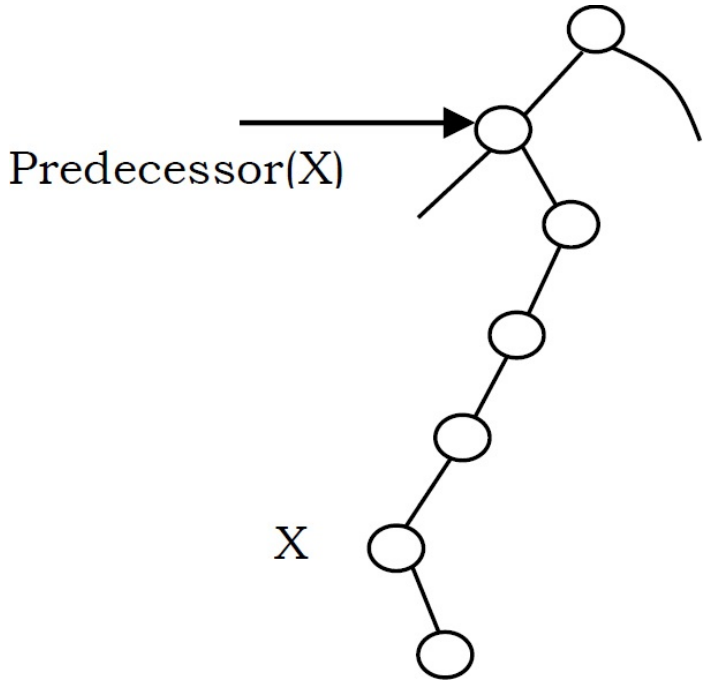
Where is Inorder Predecessor and Successor?

Where is the inorder predecessor and successor of node *X* in a binary search tree assuming all keys are distinct?

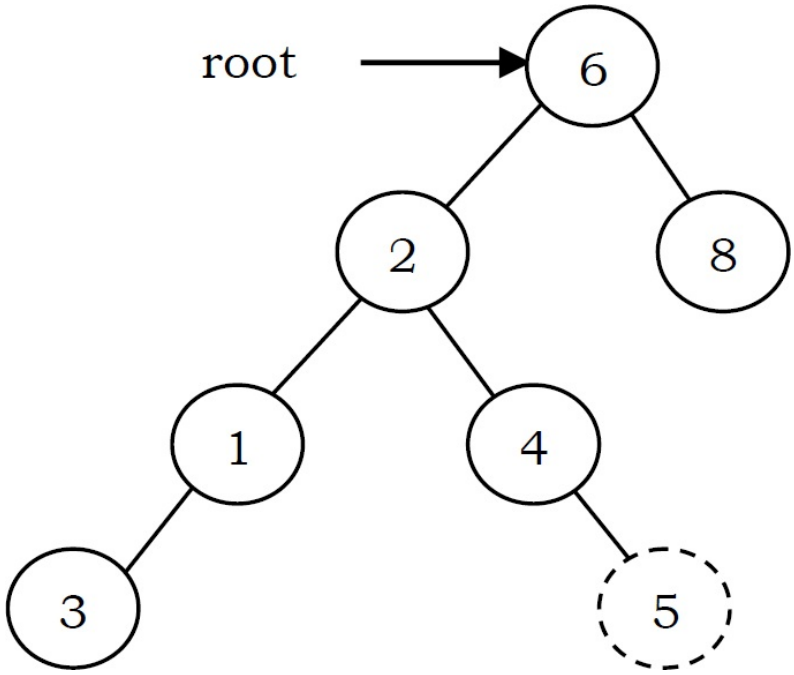
If *X* has two children then its inorder predecessor is the maximum value in its left subtree and its inorder successor the minimum value in its right subtree.



If it does not have a left child, then a node's inorder predecessor is its first left ancestor.



Inserting an Element from Binary Search Tree



To insert *data* into binary search tree, first we need to find the location for that element. We can find the location of insertion by following the same mechanism as that of *find* operation. While finding the location, if the *data* is already there then we can simply neglect and come out. Otherwise, insert *data* at the last location on the path traversed.

As an example let us consider the following tree. The dotted node indicates the element (5) to be inserted. To insert 5, traverse the tree using *find* function. At node with key 4, we need to go right, but there is no subtree, so 5 is not in the tree, and this is the correct location for insertion.

```
struct BinarySearchTreeNode *Insert(struct BinarySearchTreeNode *root, int data) {
    if( root == NULL ) {
        root = (struct BinarySearchTreeNode *) malloc(sizeof(struct  BinarySearchTreeNode));
        if( root == NULL ) {
            printf("Memory Error");
            return;
        }
        else {
            root→data = data;
            root→left = root→right = NULL;
         }
    }
    else {
        if( data < root→data )
            root→left = Insert(root→left,  data);
        else if( data > root→data )
            root→right = Insert(root→right,  data);
    }

    return root;
}
```

Note: In the above code, after inserting an element in subtrees, the tree is returned to its parent. As a result, the complete tree will get updated.

Time Complexity:O(*n*).
Space Complexity:O(*n*), for recursive stack. For iterative version, space complexity is O(1).

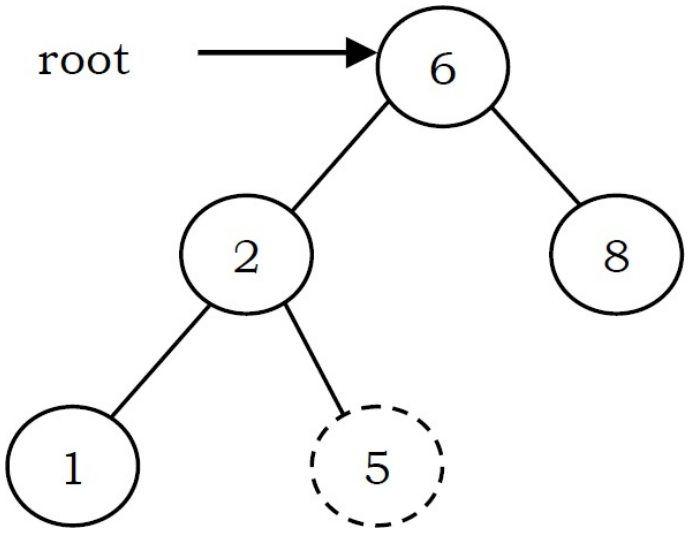
Deleting an Element from Binary Search Tree

The delete operation is more complicated than other operations. This is because the element to be deleted may not be the leaf node. In this operation also, first we need to find the location of the element which we want to delete.

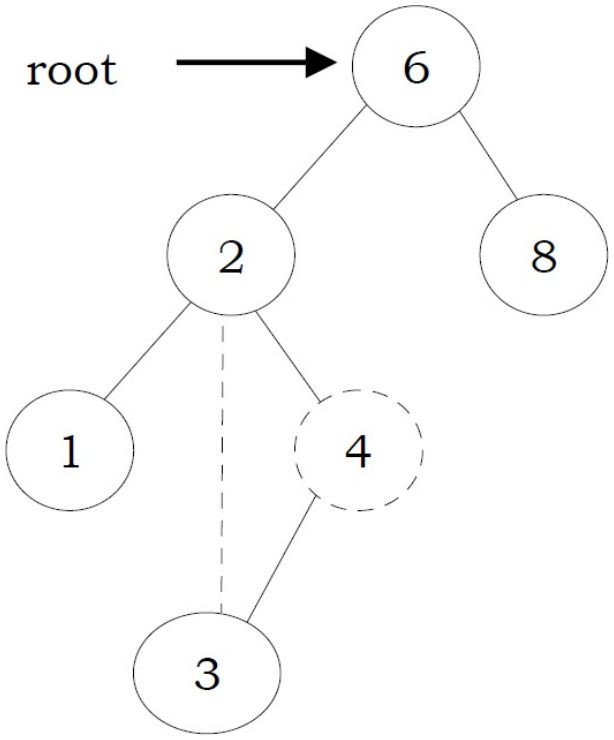
Once we have found the node to be deleted, consider the following cases:

- If the element to be deleted is a leaf node: return NULL to its parent. That means make the corresponding child pointer NULL. In the tree below to delete 5, set NULL

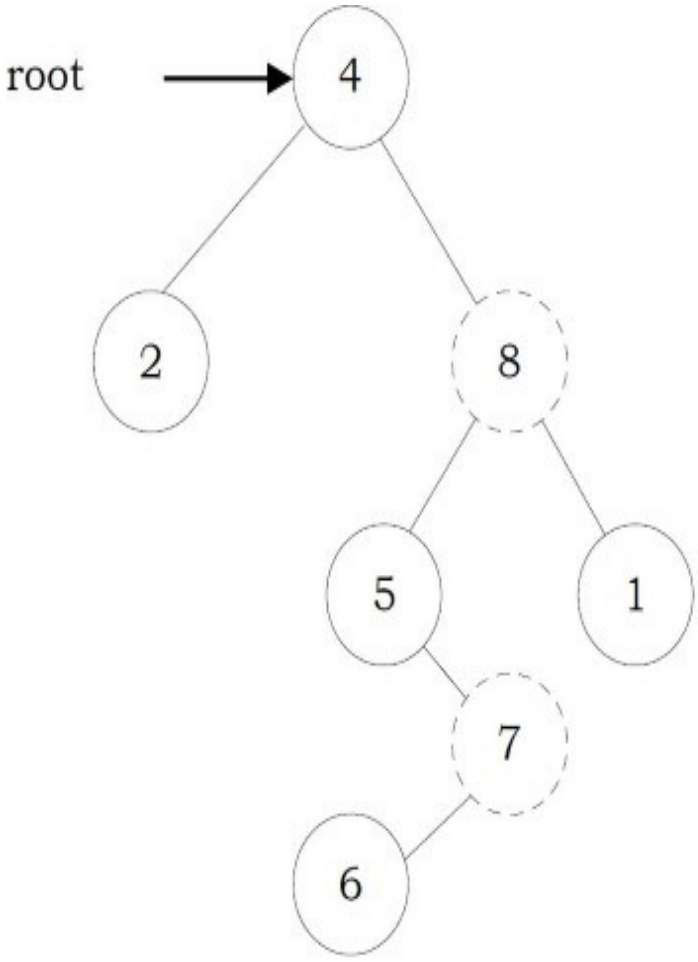
to its parent node 2.

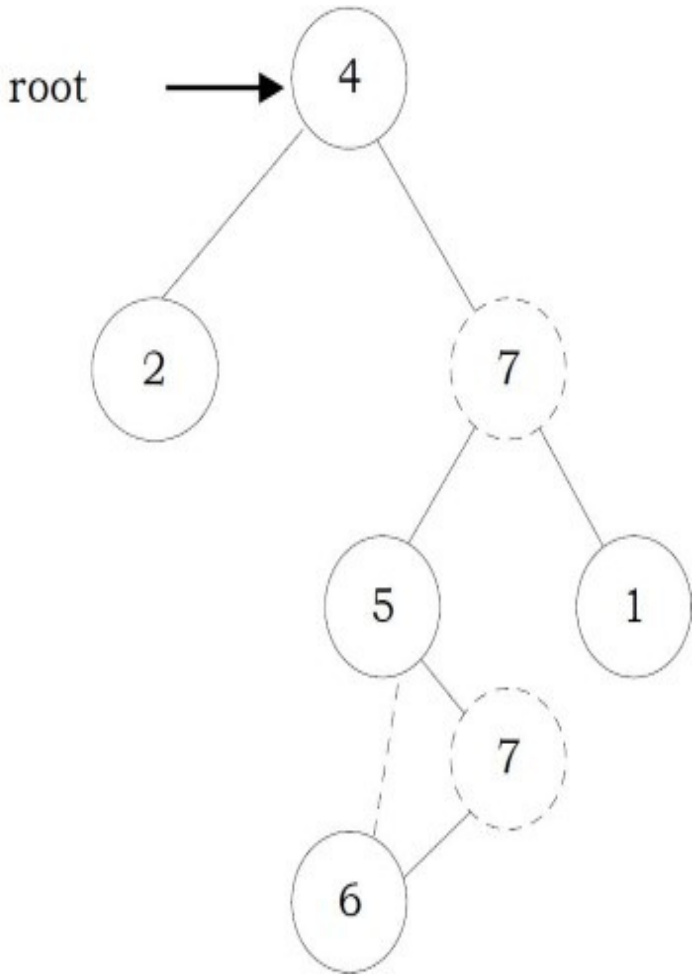


- If the element to be deleted has one child: In this case we just need to send the current node's child to its parent. In the tree below, to delete 4, 4 left subtree is set to its parent node 2.



- If the element to be deleted has both children: The general strategy is to replace the key of this node with the largest element of the left subtree and recursively delete that node (which is now empty). The largest node in the left subtree cannot have a right child, so the second *delete* is an easy one. As an example, let us consider the following tree. In the tree below, to delete 8, it is the right child of the root. The key value is 8. It is replaced with the largest key in its left subtree (7), and then that node is deleted as before (second case).





Note: We can replace with minimum element in right subtree also.

```
struct BinarySearchTreeNode *Delete(struct BinarySearchTreeNode *root, int data) {
    struct BinarySearchTreeNode *temp;
    if( root == NULL )
        printf("Element not there in tree");
    else if(data < root→data)
            root→left = Delete(root→left, data);
    else if(data > root→data )
        root→right = Delete(root→right, data);
    else {
        //Found element
        if( root→left && root→right ) {
            /* Replace with largest in left subtree */
            temp = FindMax( root→left );
            root→data = temp→data;
            root→left = Delete(root→left, root→data);
        }
        else {
                /* One child */
                temp = root;
                if( root→left == NULL )
                    root = root→right;
                if( root→right == NULL )
                    root = root→left;
                free( temp );
        }
    }
    return root;
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*) for recursive stack. For iterative version, space complexity is O(1).

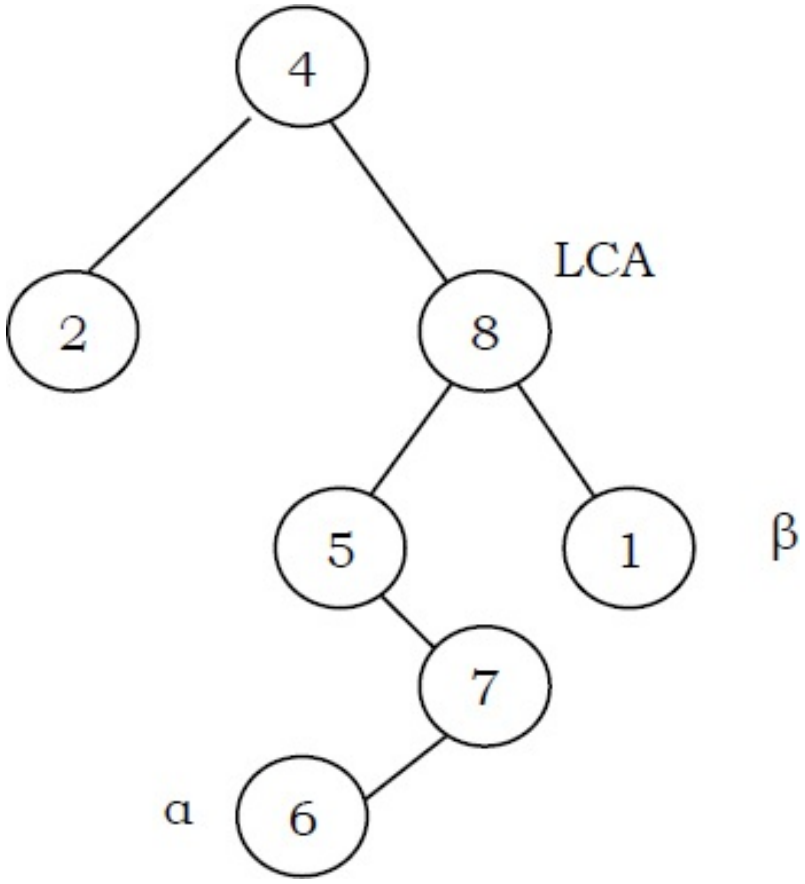
Binary Search Trees: Problems & Solutions

Note: For ordering related problems with binary search trees and balanced binary search trees,

Inorder traversal has advantages over others as it gives the sorted order.

Problem-47 Given pointers to two nodes in a binary search tree, find the lowest common ancestor (*LCA*). Assume that both values already exist in the tree.

Solution:



The main idea of the solution is: while traversing BST from root to bottom, the first node we encounter with value between α and β, i.e., $\alpha < node \rightarrow data < \beta$, is the Least Common Ancestor(LCA) of α and β (where $\alpha < \beta$). So just traverse the BST in pre-order, and if we find a node with value in between α and β, then that node is the LCA. If its value is greater than both α and β, then the LCA lies on the left side of the node, and if its value is smaller than both α and β, then the LCA lies on the right side.

```
struct BinarySearchTreeNode *FindLCA(struct BinarySearchTreeNode *root, struct BinarySearchTreeNode *α,
                                     struct BinarySearchTreeNode * β) {
    while(1) {
        if((α→data < root→data && β→data > root→data) ||
                                  (α→data > root→data && ß→data < root→data))
            return root;
        if(α→data < root→data)
          root = root→left;
        else root = root→right;
    }
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*), for skew trees.

Problem-48 Give an algorithm for finding the shortest path between two nodes in a BST.

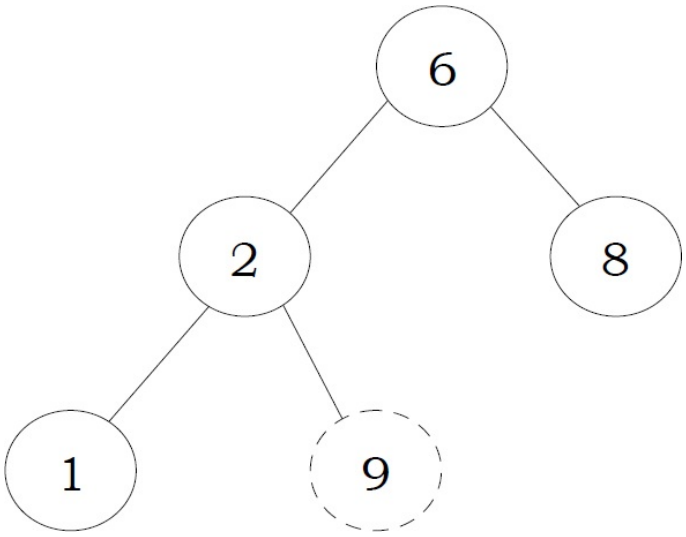
Solution: It's nothing but finding the LCA of two nodes in BST.

Problem-49 Give an algorithm for counting the number of BSTs possible with *n* nodes.

Solution: This is a DP problem. Refer to chapter on *Dynamic Programming* for the algorithm.

Problem-50 Give an algorithm to check whether the given binary tree is a BST or not.

Solution:



Consider the following simple program. For each node, check if the node on its left is smaller and check if the node on its right is greater. This approach is wrong as this will return true for binary tree below. Checking only at current node is not enough.

```
int IsBST(struct BinaryTreeNode* root) {
    if(root == NULL)
        return 1;
    // false if left is > than root
    if(root→left != NULL && root→left→data > root→data)
        return 0;
    // false if right is < than root
    if(root→right != NULL && root→right→data < root→data)
        return 0;
    // false if, recursively, the left or right is not a BST
    if(!IsBST(root→left) || !IsBST(root→right))
        return 0;
    // passing all that, it's a BST
    return 1;
}
```

Problem-51 Can we think of getting the correct algorithm?

Solution: For each node, check if max value in left subtree is smaller than the current node data and min value in right subtree greater than the node data. It is assumed that we have helper functions *FindMin*() and *FindMax*() that return the min or max integer value from a non-empty tree.

```
/* Returns true if a binary tree is a binary search tree */
int IsBST(struct BinaryTreeNode* root) {
    if(root == NULL)
        return 1;
    /* false if the max of the left is > than root */
    if(root→left != NULL && FindMax(root→left) > root→data)
        return 0;
    /* false if the min of the right is <= than root */
    if(root→right != NULL && FindMin(root→right) < root→data)
        return 0;
    /* false if, recursively, the left or right is not a BST */
    if(!IsBST(root→left) || !IsBST(root→right))
        return 0;
    /* passing all that, it's a BST */
    return 1;
}
```

Time Complexity: O(n^2). Space Complexity: O(n).

Problem-52 Can we improve the complexity of Problem-51?

Solution: Yes. A better solution is to look at each node only once. The trick is to write a utility helper function IsBSTUtil(struct BinaryTreeNode* root, int min, int max) that traverses down the tree keeping track of the narrowing min and max allowed values as it goes, looking at each node only once. The initial values for min and max should be INT_MIN and INT_MAX – they narrow from there.

```
Initial call: IsBST(root, INT_MIN, INT_MAX);
int IsBST(struct BinaryTreeNode *root, int min, int max) {
      if(!root)
        return 1;
      return (root→data >min && root→data < max &&
              IsBSTUtil(root→left, min, root→data) &&
              IsBSTUtil(root→right, root→data, max));
}
```

Time Complexity: O(n). Space Complexity: O(n), for stack space.

Problem-53 Can we further improve the complexity of Problem-51?

Solution: Yes, by using inorder traversal. The idea behind this solution is that inorder traversal of BST produces sorted lists. While traversing the BST in inorder, at each node check the condition that its key value should be greater than the key value of its previous visited node. Also, we need to initialize the prev with possible minimum integer value (say, INT_MIN).

```
int prev = INT_MIN;
int IsBST(struct BinaryTreeNode *root, int *prev) {
 if(!root) return 1;
 if(!IsBST(root→left, prev))
      return 0;
 if(root→data < *prev)
      return 0;
 *prev = root→data;
 return IsBST(root→right, prev);
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*), for stack space.

Problem-54 Give an algorithm for converting BST to circular DLL with space complexity O(1).

Solution: Convert left and right subtrees to DLLs and maintain end of those lists. Then, adjust the pointers.

```
struct BinarySearchTreeNode *BST2DLL(struct BinarySearchTreeNode *root,
                                     struct BinarySearchTreeNode **Ltail) {
    struct BinarySearchTreeNode *left, *ltail, *right, *rtail;
    if(!root) {
        * ltail = NULL;
        return NULL;
    }
    left = BST2DLL(root→left, &ltail);
    right = BST2DLL(root→right, &rtail);
    root→left = ltail;
    root→right = right;
    if(!right)
        * ltail = root;
    else {
        right→left = root;
        * ltail = rtail;
    }
    if(!left)
        return root;
    else {
        ltail→right = root;
        return left;
    }
}
```

Time Complexity: O(*n*).

Problem-55 For Problem-54, is there any other way of solving it?

Solution: Yes. There is an alternative solution based on the divide and conquer method which is quite neat.

```
struct BinarySearchTreeNode *Append(struct BinarySearchTreeNode *a, struct BinarySearchTreeNode *b) {
    struct BinarySearchTreeNode  *aLast, *bLast;
    if (a==NULL)
        return b;
    if (b==NULL)
        return a;
    aLast = a→left;
    bLast = b→left;
    aLast→right = b;
    b→left = aLast;
    bLast→right = a;
    a→left = bLast;
    return a;
}
struct BinarySearchTreeNode* TreeToList(struct BinarySearchTreeNode *root) {
    struct BinarySearchTreeNode *aList, *bList;
    if (root==NULL)
      return NULL;
    aList = TreeToList(root→left);
    bList = TreeToList(root→right);
    root→left = root;
    root→right = root;
    aList = Append(aList, root);
    aList = Append(aList, bList);
    return(aList);
}
```

Time Complexity: O(*n*).

Problem-56 Given a sorted doubly linked list, give an algorithm for converting it into balanced binary search tree.

Solution: Find the middle node and adjust the pointers.

```
struct DLLNode * DLLtoBalancedBST(struct DLLNode *head) {
    struct DLLNode *temp, *p, *q;
    if( !head || !head→next)
        return head;
    temp = FindMiddleNode(head);
    p = head;
    while(p→next != temp)
        p = p→next;
    p→next = NULL;
    q = temp→next;
    temp→next = NULL;
    temp→prev = DLLtoBalancedBST(head);
    temp→next = DLLtoBalancedBST(q);
    return temp;
}
```

Time Complexity: $2T(n/2) + O(n)$ [for finding the middle node] = $O(nlogn)$.

Note: For *FindMiddleNode* function refer *Linked Lists* chapter.

Problem-57 Given a sorted array, give an algorithm for converting the array to BST.

Solution: If we have to choose an array element to be the root of a balanced BST, which element should we pick? The root of a balanced BST should be the middle element from the sorted array. We would pick the middle element from the sorted array in each iteration. We then create a node in the tree initialized with this element. After the element is chosen, what is left? Could you identify the sub-problems within the problem?

There are two arrays left – the one on its left and the one on its right. These two arrays are the sub-problems of the original problem, since both of them are sorted. Furthermore, they are subtrees of the current node's left and right child.

The code below creates a balanced BST from the sorted array in $O(n)$ time (n is the number of elements in the array). Compare how similar the code is to a binary search algorithm. Both are using the divide and conquer methodology.

```
struct BinaryTreeNode  *BuildBST(int A[], int left, int right) {
    struct BinaryTreeNode  *newNode;
    int mid;
    if(left > right)
        return NULL;
    newNode = (struct BinaryTreeNode  *)malloc(sizeof(struct BinaryTreeNode));
    if(!newNode) {
      printf("Memory Error");
      return;
    }
    if(left == right) {
      newNode→data = A[left];
      newNode→left = newNode→right = NULL;
    }
    else {    mid = left + (right-left)/ 2;
      newNode→data = A[mid];
      newNode→left = BuildBST(A, left, mid - 1);
      newNode→right = BuildBST(A, mid + 1, right);
    }
    return newNode;
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*), for stack space.

Problem-58 Given a singly linked list where elements are sorted in ascending order, convert it to a height balanced BST.

Solution: A naive way is to apply the Problem-56 solution directly. In each recursive call, we would have to traverse half of the list's length to find the middle element. The run time complexity is clearly O(*nlogn*), where *n* is the total number of elements in the list. This is because each level of recursive call requires a total of *n*/2 traversal steps in the list, and there are a total of *logn* number of levels (ie, the height of the balanced tree).

Problem-59 For Problem-58, can we improve the complexity?

Solution: Hint: How about inserting nodes following the list order? If we can achieve this, we no longer need to find the middle element as we are able to traverse the list while inserting nodes to the tree.

Best Solution: As usual, the best solution requires us to think from another perspective. In other words, we no longer create nodes in the tree using the top-down approach. Create nodes bottom-up, and assign them to their parents. The bottom-up approach enables us to access the list in its order while creating nodes [42].

Isn't the bottom-up approach precise? Any time we are stuck with the top-down approach, we can give bottom-up a try. Although the bottom-up approach is not the most natural way we think, it is helpful in some cases. However, we should prefer top-down instead of bottom-up in general, since the latter is more difficult to verify.

Below is the code for converting a singly linked list to a balanced BST. Please note that the algorithm requires the list length to be passed in as the function parameters. The list length can be found in O(*n*) time by traversing the entire list once. The recursive calls traverse the list and create tree nodes by the list order, which also takes O(*n*) time. Therefore, the overall run time complexity is still O(*n*).

```
struct BinaryTreeNode* SortedListToBST(struct ListNode *& list, int start, int end) {
    if(start > end)
        return NULL;
    // same as (start+end)/2, avoids overflow
    int mid = start + (end - start) / 2;
    struct BinaryTreeNode *leftChild = SortedListToBST(list, start, mid-1);
    struct BinaryTreeNode  * parent;
    parent = (struct BinaryTreeNode  *)malloc(sizeof(struct BinaryTreeNode));
    if(!parent) {
      printf("Memory Error");
      return;
    }
    parent→data=list→data;
    parent→left = leftChild;
    list = list→next;
    parent→right = SortedListToBST(list, mid+1, end);
    return parent;
}
struct BinaryTreeNode * SortedListToBST(struct ListNode *head, int n) {
    return SortedListToBST(head, 0, n-1);
}
```

Problem-60 Give an algorithm for finding the k^{th} smallest element in BST.

Solution: The idea behind this solution is that, inorder traversal of BST produces sorted lists. While traversing the BST in inorder, keep track of the number of elements visited.

```
struct BinarySearchTreeNode *kthSmallestInBST(struct BinarySearchTreeNode *root, int k, int *count){
    if(!root)
        return NULL;
    struct BinarySearchTreeNode *left = kthSmallestInBST(root→left, k, count);
    if( left )
        return left;
    if(++count == k)
        return root;
    return kthSmallestInBST(root→right, k, count);
}
```

Time Complexity: O(*n*). Space Complexity: O(1).

Problem-61 **Floor and ceiling:** If a given key is less than the key at the root of a BST then the floor of the key (the largest key in the BST less than or equal to the key) must be in the left subtree. If the key is greater than the key at the root, then the floor of the key could be in the right subtree, but only if there is a key smaller than or equal to the key in the right subtree; if not (or if the key is equal to the the key at the root) then the key at the root is the floor of the key. Finding the ceiling is similar, with interchanging right and left. For example, if the sorted with input array is {1, 2, 8, 10, 10, 12, 19}, then

For $x = 0$: floor doesn't exist in array, ceil = 1, For $x = 1$: floor = 1, ceil = 1
For $x = 5$: floor =2, ceil = 8, For $x = 20$: floor = 19, ceil doesn't exist in array

Solution: The idea behind this solution is that, inorder traversal of BST produces sorted lists. While traversing the BST in inorder, keep track of the values being visited. If the roots data is greater than the given value then return the previous value which we have maintained during traversal. If the roots data is equal to the given data then return root data.

```
struct BinaryTreeNode *FloorInBST(struct BinaryTreeNode *root, int data){
    struct BinaryTreeNode *prev=NULL;
    return FloorInBSTUtil(root, prev, data);
}
struct BinaryTreeNode *FloorInBSTUtil(struct BinaryTreeNode *root,
                                      struct BinaryTreeNode *prev, int data){
    if(!root)
        return NULL;
    if(!FloorInBSTUtil(root→left, prev, data))
        return 0;
    if(root→data == data)
        return root;
    if(root→data > data)
        return prev;
    prev = root;
    return FloorInBSTUtil(root→right, prev, data);
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*), for stack space.

For ceiling, we just need to call the right subtree first, followed by left subtree.

```
struct BinaryTreeNode *CeilingInBST(struct BinaryTreeNode *root, int data){
    struct BinaryTreeNode *prev=NULL;
    return CeilingInBSTUtil(root, prev, data);
}

struct BinaryTreeNode *CeilingInBSTUtil(struct BinaryTreeNode *root,
                                        struct BinaryTreeNode *prev, int data){
    if(!root)
        return NULL;
    if(!CeilingInBSTUtil(root→right, prev, data))
        return 0;
    if(root→data == data)
        return root;
    if(root→data < data)
        return prev;
    prev = root;
    return CeilingInBSTUtil(root→left, prev, data);
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*), for stack space.

Problem-62 Give an algorithm for finding the union and intersection of BSTs. Assume parent pointers are available (say threaded binary trees). Also, assume the lengths of two BSTs are *m* and n respectively.

Solution: If parent pointers are available then the problem is same as merging of two sorted lists. This is because if we call inorder successor each time we get the next highest element. It's just a matter of which InorderSuccessor to call.

Time Complexity: O(*m* + *n*). Space complexity: O(1).

Problem-63 For Problem-62, what if parent pointers are not available?

Solution: If parent pointers are not available, the BSTs can be converted to linked lists and then merged.

1. Convert both the BSTs into sorted doubly linked lists in O(*n* + *m*) time. This produces 2 sorted lists.
2. Merge the two double linked lists into one and also maintain the count of total elements in O(*n* + *m*) time.
3. Convert the sorted doubly linked list into height balanced tree in O(*n* + *m*) time.

Problem-64 For Problem-62, is there any alternative way of solving the problem?

Solution: Yes, by using inorder traversal.

- Perform inorder traversal on one of the BSTs.
- While performing the traversal store them in table (hash table).
- After completion of the traversal of first *BST*, start traversal of second *BST* and compare them with hash table contents.

Time Complexity: O($m + n$). Space Complexity: O($Max(m,n)$).

Problem-65 Given a *BST* and two numbers $K1$ and $K2$, give an algorithm for printing all the elements *of BST* in the range $K1$ and $K2$.

Solution:

```
void RangePrinter(struct BinarySearchTreeNode *root, int  K1, int K2) {
    if(root == NULL)
        return;
    if(root→data >= K1)
        RangePrinter(root→left, K1, K2);
    if(root→data >= K1 && root→data <= K2)
            printf("%d", root→data);
    if(root→data <= K2)
        RangePrinter(root→right, K1, K2);
}
```

Time Complexity: O(n). Space Complexity: O(n), for stack space.

Problem-66 For Problem-65, is there any alternative way of solving the problem?

Solution: We can use level order traversal: while adding the elements to queue check for the range.

```
void RangeSeachLevelOrder(struct BinarySearchTreeNode *root, int K1, int K2){
    struct BinarySearchTreeNode *temp;
    struct Queue *Q = CreateQueue();
    if(!root)
        return NULL;
    Q = EnQueue(Q, root);
    while(!IsEmptyQueue(Q)) {
            temp=DeQueue(Q);
            if(temp→data >= K1 && temp→data <= K2)
                    printf("%d",temp→data);
            if(temp→left && temp→data >= K1)
                    EnQueue(Q, temp→left);
            if(temp→right && temp→data <= K2)
                    EnQueue(Q, temp→right);
    }
    DeleteQueue(Q);
    return NULL;
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*), for queue.

Problem-67 For Problem-65, can we still think of an alternative way to solve the problem?

Solution: First locate *K*1 with normal binary search and after that use InOrder successor until we encounter *K*2. For algorithm, refer to problems section of threaded binary trees.

Problem-68 Given root of a Binary Search tree, trim the tree, so that all elements returned in the new tree are between the inputs *A* and *B*.

Solution: It's just another way of asking Problem-65.

Problem-69 Given two BSTs, check whether the elements of them are the same or not. For example: two BSTs with data 10 5 20 15 30 and 10 20 15 30 5 should return true and the dataset with 10 5 20 15 30 and 10 15 30 20 5 should return false. **Note:** BSTs data can be in any order.

Solution: One simple way is performing an inorder traversal on first tree and storing its data in hash table. As a second step, perform inorder traversal on second tree and check whether that data is already there in hash table or not (if it exists in hash table then mark it with -1 or some unique value).

During the traversal of second tree if we find any mismatch return false. After traversal of second tree check whether it has all -1s in the hash table or not (this ensures extra data available in second tree).

Time Complexity: O(*max*(*m, n*)), where *m* and n are the number of elements in first and second BST. Space Complexity: O(*max*(*m,n*)). This depends on the size of the first tree.

Problem-70 For Problem-69, can we reduce the time complexity?

Solution: Instead of performing the traversals one after the other, we can perform *in – order* traversal of both the trees in parallel. Since the *in – order* traversal gives the sorted list, we can check whether both the trees are generating the same sequence or not.

Time Complexity: O(*max*(*m,n*)). Space Complexity: O(1). This depends on the size of the first tree.

Problem-71 For the key values 1... *n*, how many structurally unique BSTs are possible that store those keys.

Solution: Strategy: consider that each value could be the root. Recursively find the size of the left and right subtrees.

```
int CountTrees(int n) {
    if (n <= 1)
        return 1;
    else {
        // there will be one value at the root, with whatever remains on the left and right
        // each forming their own subtrees. Iterate through all the values that could be the root...
        int sum = 0;
        int left, right, root;
        for (root=1; root<=n; root++) {
            left = CountTrees(root - 1);
            right = CountTrees(numKeys - root);

            // number of possible trees with this root == left*right
            sum += left*right;
        }
        return(sum);
    }
}
```

Problem-72 Given a BST of size *n*, in which each node r has an additional field $r \rightarrow size$,

the number of the keys in the sub-tree rooted at *r* (including the root node *r*). Give an O(*h*) algorithm *GreaterthanConstant*(*r,k*) to find the number of keys that are strictly greater than *k* (*h* is the height of the binary search tree).

Solution:

```
int GreaterthanConstant (struct BinarySearchTreeNode *r, int k){
   keysCount = 0
   while (r != Null ){
      if (k < r→data){
         keysCount = keysCount + r→right→size + 1;
         r = r→left;
      }
      else if (k > r→data)
         r = r→right;
      else{ // k = r→key
         keysCount = keysCount + r→right→size;
         break;
      }
   }
   return keysCount;
}
```

The suggested algorithm works well if the key is a unique value for each node. Otherwise when reaching $k=r \rightarrow data$, we should start a process of moving to the right until reaching a node y with a key that is bigger then k, and then we should return $keysCount + y \rightarrow size$. Time Complexity: O(*h*) where $h=O(n)$ in the worst case and O(*logn*) in the average case.

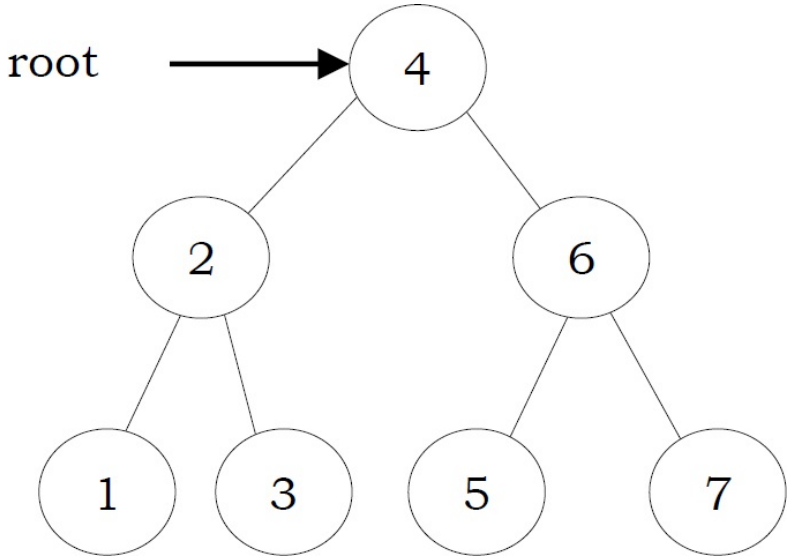
6.12 Balanced Binary Search Trees

In earlier sections we have seen different trees whose worst case complexity is O(*n*), where n is the number of nodes in the tree. This happens when the trees are skew trees. In this section we will try to reduce this worst case complexity to O(*logn*) by imposing restrictions on the heights.

In general, the height balanced trees are represented with *HB*(*k*), where *k* is the difference between left subtree height and right subtree height. Sometimes *k* is called balance factor.

Full Balanced Binary Search Trees

In *HB*(*k*), if *k* = 0 (if balance factor is zero), then we call such binary search trees as *full* balanced binary search trees. That means, in *HB*(0) binary search tree, the difference between left subtree height and right subtree height should be at most zero. This ensures that the tree is a full binary tree. For example,



Note: For constructing *HB*(0) tree refer to *Problems* section.

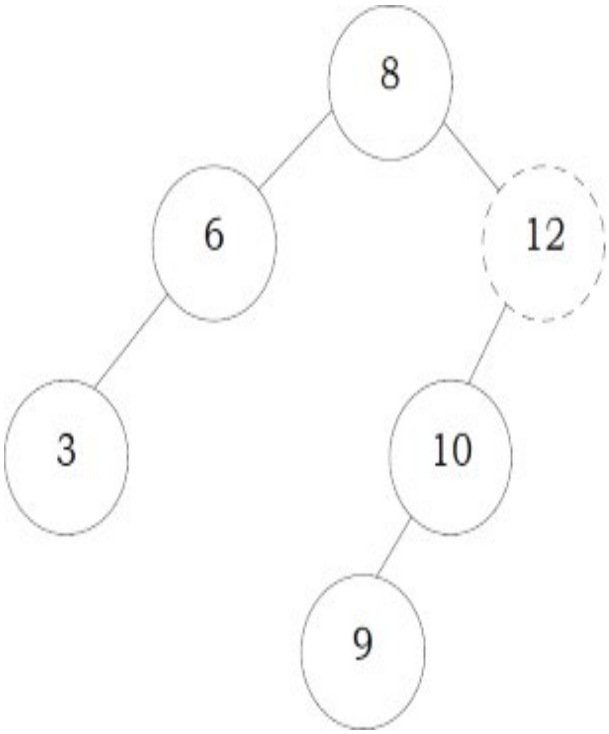
6.13 AVL (Adelson-Velskii and Landis) Trees

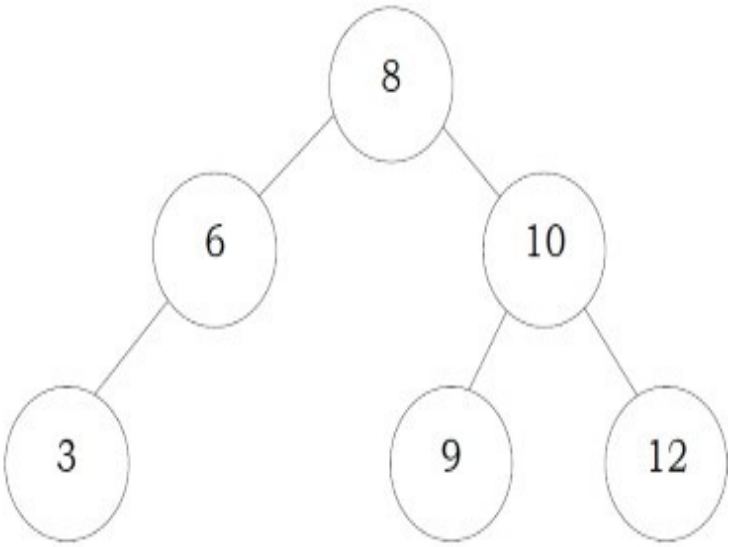
In *HB*(*k*), if *k* = 1 (if balance factor is one), such a binary search tree is called an *AVL tree*. That means an AVL tree is a binary search tree with a *balance* condition: the difference between left subtree height and right subtree height is at most 1.

Properties of AVL Trees

A binary tree is said to be an AVL tree, if:

- It is a binary search tree, and
- For any node *X*, the height of left subtree of *X* and height of right subtree of *X* differ by at most 1.





As an example, among the above binary search trees, the left one is not an AVL tree, whereas the right binary search tree is an AVL tree.

Minimum/Maximum Number of Nodes in AVL Tree

For simplicity let us assume that the height of an AVL tree is h and $N(K)$ indicates the number of nodes in AVL tree with height h. To get the minimum number of nodes with height h, we should fill the tree with the minimum number of nodes possible. That means if we fill the left subtree with height $h - 1$ then we should fill the right subtree with height $h - 2$. As a result, the minimum number of nodes with height h is:

$$N(h) = N(h - 1) + N(h - 2) + 1$$

In the above equation:

- $N(h - 1)$ indicates the minimum number of nodes with height $h - 1$.
- $N(h - 2)$ indicates the minimum number of nodes with height $h - 2$.
- In the above expression, “1” indicates the current node.

We can give $N(h - 1)$ either for left subtree or right subtree. Solving the above recurrence gives:

$$N(h) = O(1.618^h) \Rightarrow h = 1.44 logn \approx O(logn)$$

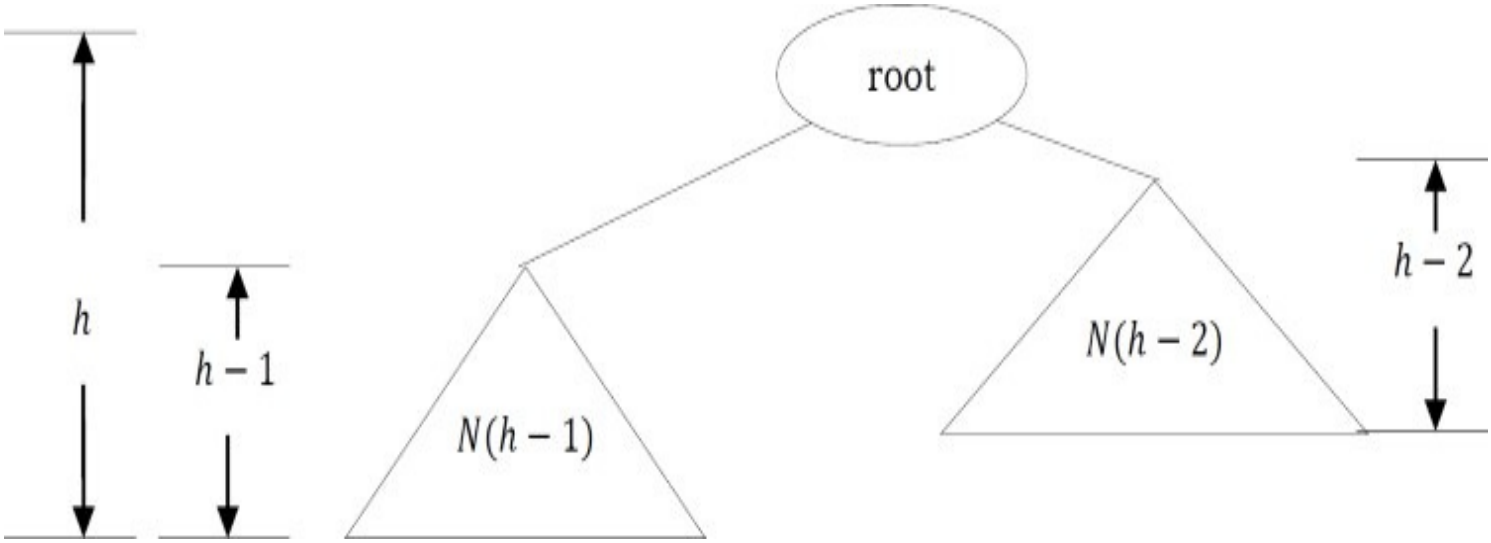
Where n is the number of nodes in AVL tree. Also, the above derivation says that the maximum height in AVL trees is $O(logn)$. Similarly, to get maximum number of nodes, we need to fill both left and right subtrees with height $h - 1$. As a result, we get:

$$N(h) = N(h-1) + N(h-1) + 1 = 2N(h-1) + 1$$

The above expression defines the case of full binary tree. Solving the recurrence we get:

$$N(h) = O(2^h) \Rightarrow h = logn \approx O(logn)$$

∴ In both the cases, AVL tree property is ensuring that the height of an AVL tree with n nodes is $O(logn)$.



AVL Tree Declaration

Since AVL tree is a BST, the declaration of AVL is similar to that of BST. But just to simplify the operations, we also include the height as part of the declaration.

```
struct AVLTreeNode{
    struct AVLTreeNode *left;
    int data;
    struct AVLTreeNode *right;
    int height;
};
```

Finding the Height of an AVL tree

```
int Height(struct AVLTreeNode *root ){
    if( !root)
        return -1;
    else
        return root→height;
}
```

Time Complexity: O(1).

Rotations

When the tree structure changes (e.g., with insertion or deletion), we need to modify the tree to restore the AVL tree property. This can be done using single rotations or double rotations. Since an insertion/deletion involves adding/deleting a single node, this can only increase/decrease the height of a subtree by 1.

So, if the AVL tree property is violated at a node *X*, it means that the heights of left(*X*) and right(*X*) differ by exactly 2. This is because, if we balance the AVL tree every time, then at any point, the difference in heights of left(*X*) and right(*X*) differ by exactly 2. Rotations is the technique used for restoring the AVL tree property. This means, we need to apply the rotations for the node *X*.

Observation: One important observation is that, after an insertion, only nodes that are on the path from the insertion point to the root might have their balances altered, because only those nodes have their subtrees altered. To restore the AVL tree property, we start at the insertion point and keep going to the root of the tree.

While moving to the root, we need to consider the first node that is not satisfying the AVL property. From that node onwards, every node on the path to the root will have the issue.

Also, if we fix the issue for that first node, then all other nodes on the path to the root will automatically satisfy the AVL tree property. That means we always need to care for the first node that is not satisfying the AVL property on the path from the insertion point to the root and fix it.

Types of Violations

Let us assume the node that must be rebalanced is *X*. Since any node has at most two children, and a height imbalance requires that *X's* two subtree heights differ by two, we can observe that a violation might occur in four cases:

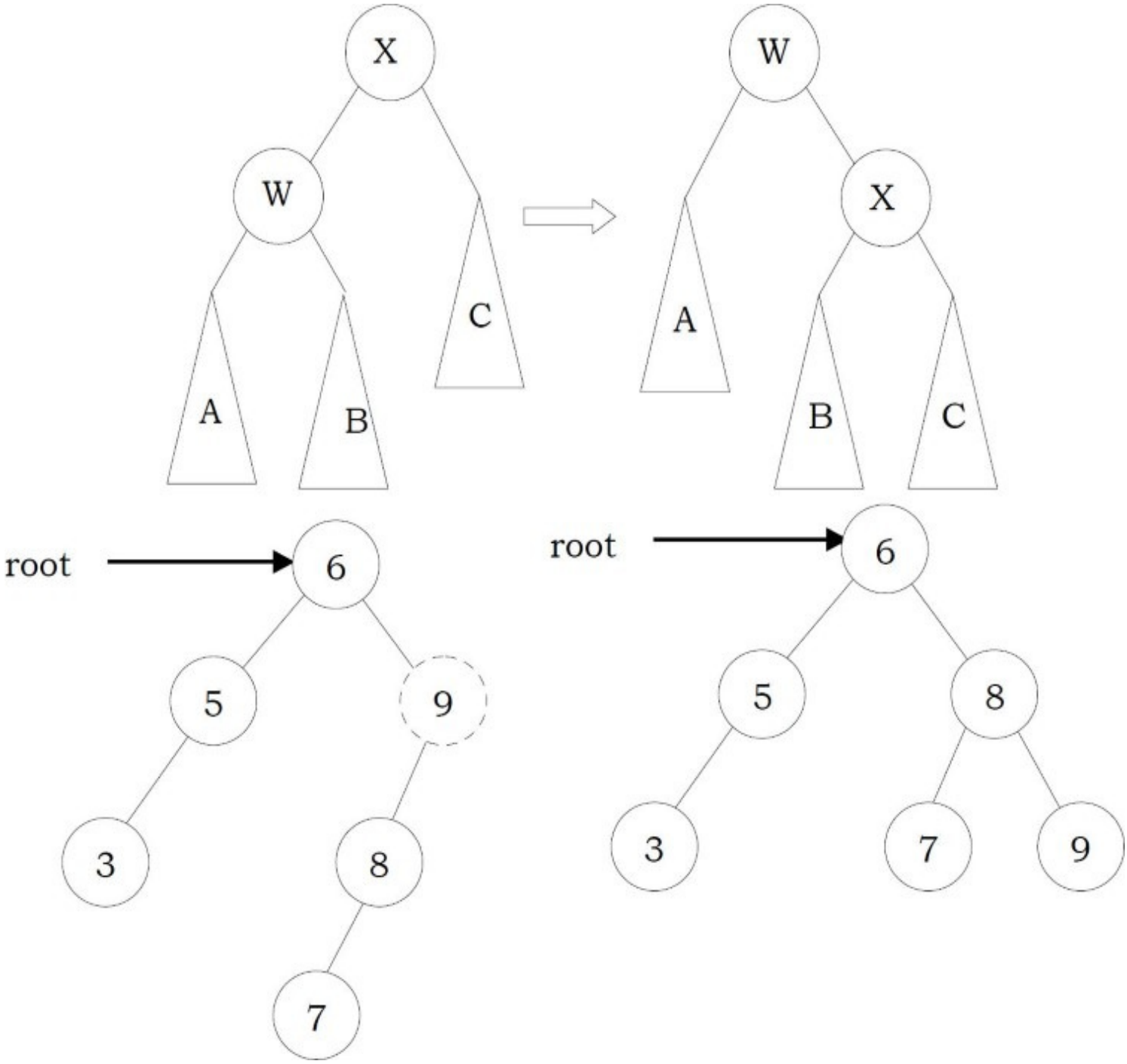
1. An insertion into the left subtree of the left child of *X*.
2. An insertion into the right subtree of the left child of *X*.

3. An insertion into the left subtree of the right child of *X*.
4. An insertion into the right subtree of the right child of *X*.

Cases 1 and 4 are symmetric and easily solved with single rotations. Similarly, cases 2 and 3 are also symmetric and can be solved with double rotations (*n*eeds two single rotations).

Single Rotations

Left Left Rotation (LL Rotation) [Case-1]: In the case below, node *X* is not satisfying the AVL tree property. As discussed earlier, the rotation does not have to be done at the root of a tree. In general, we start at the node inserted and travel up the tree, updating the balance information at every node on the path.

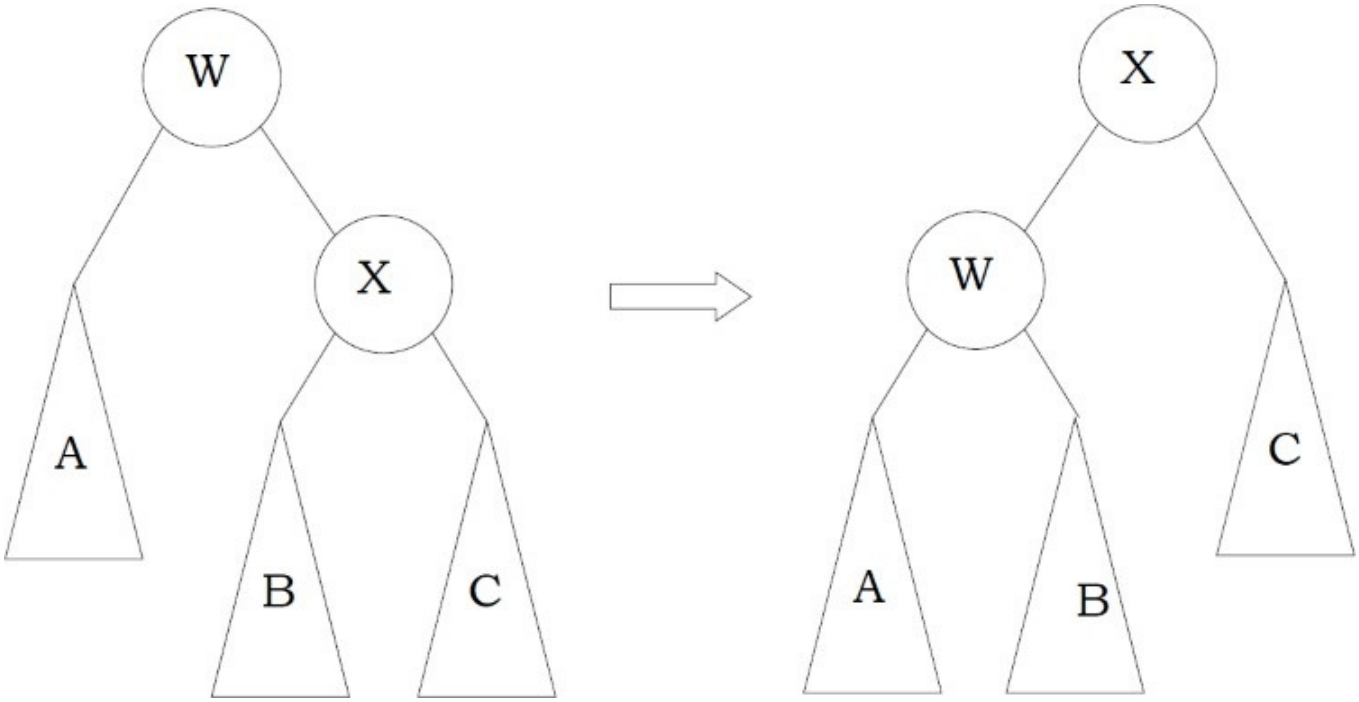


For example, in the figure above, after the insertion of 7 in the original AVL tree on the left, node 9 becomes unbalanced. So, we do a single left-left rotation at 9. As a result we get the tree on the right.

```
struct AVLTreeNode *SingleRotateLeft(struct AVLTreeNode *X ){
    struct AVLTreeNode *W = X→left;
    X→left = W→right;
    W→right = X;
    X→height = max( Height(X→left), Height(X→right) ) + 1;
    W→height = max( Height(W→left), X→height ) + 1;
    return W;  /* New root */
}
```

Time Complexity: O(1). Space Complexity: O(1).

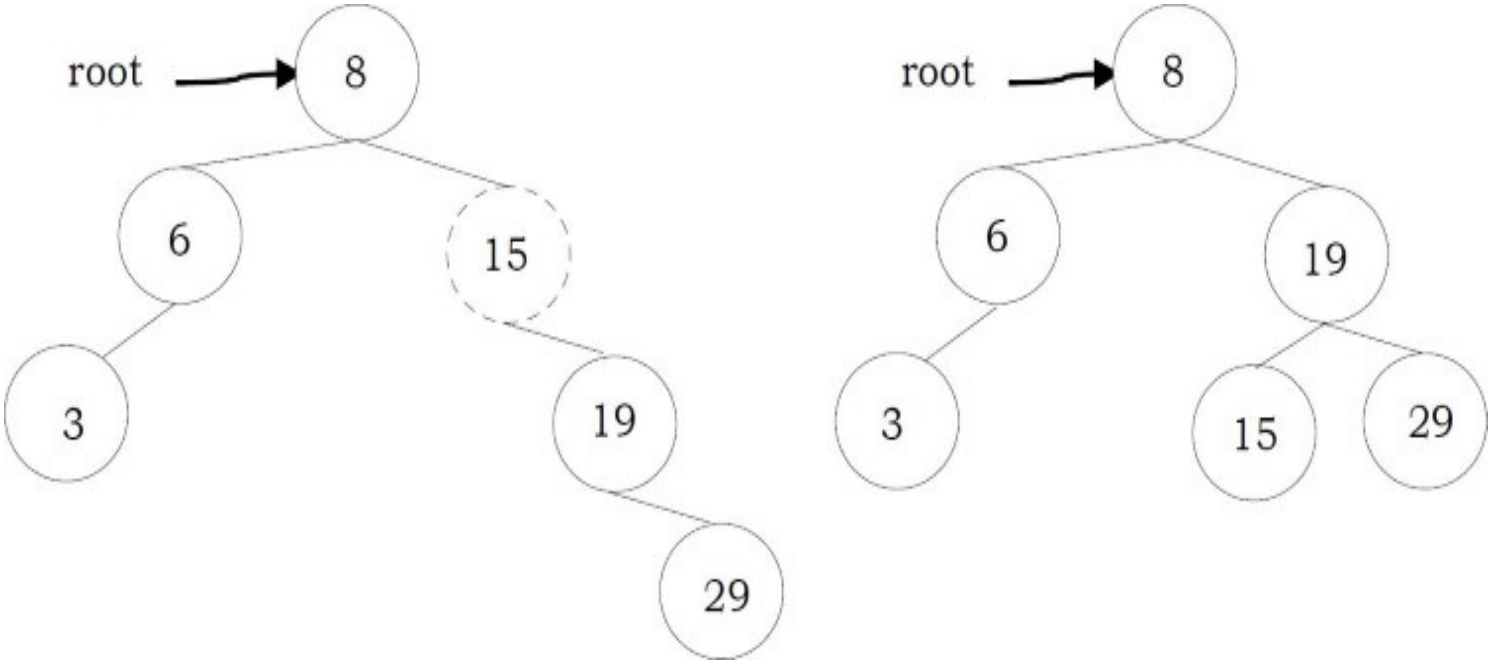
Right Right Rotation (RR Rotation) [Case-4]: In this case, node *X* is not satisfying the AVL tree property.



For example, in the figure, after the insertion of 29 in the original AVL tree on the left, node 15 becomes unbalanced. So, we do a single right-right rotation at 15. As a result we get the tree on the right.

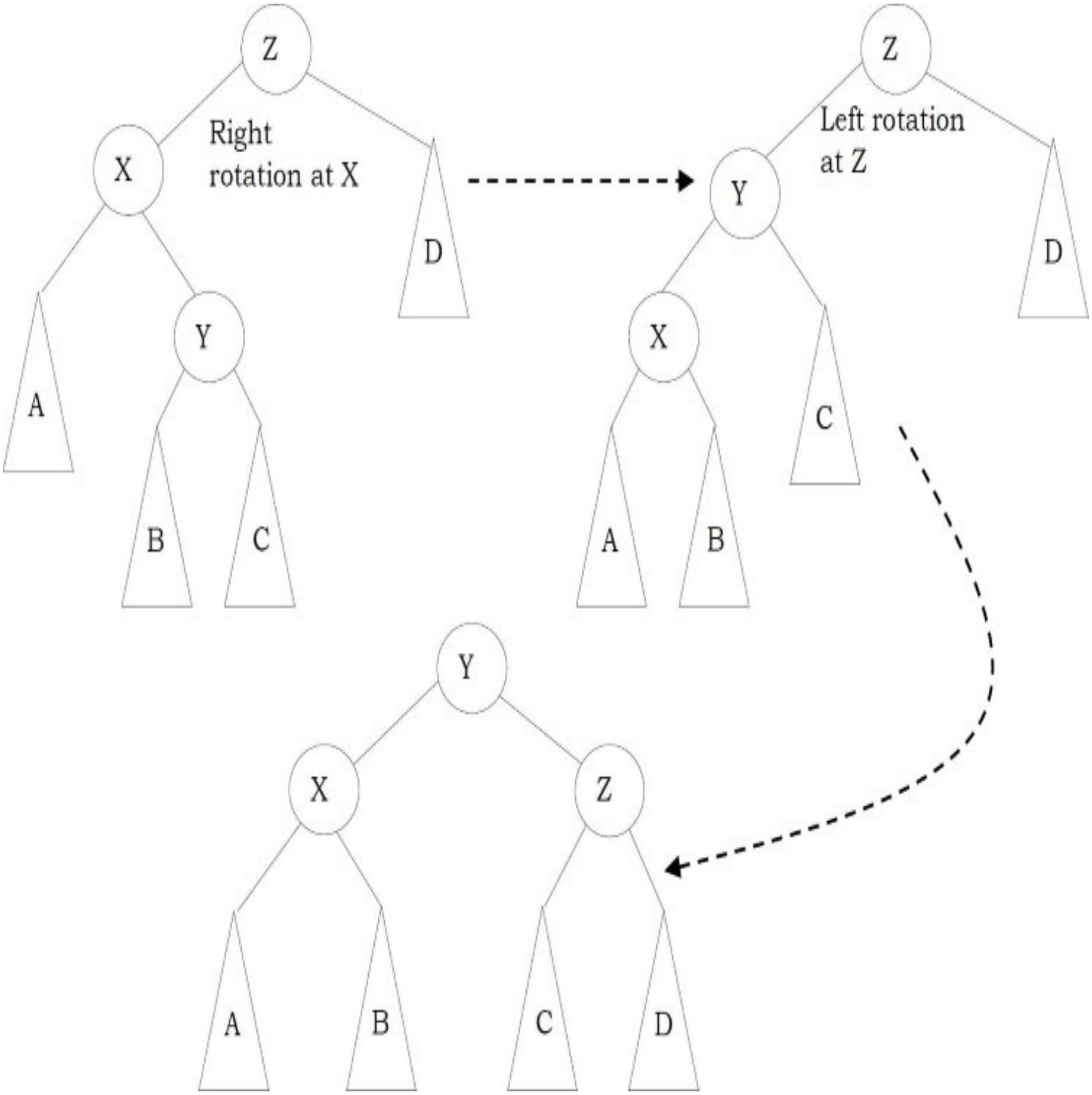
```
struct AVLTreeNode *SingleRotateRight(struct AVLTreeNode *W ) {
    struct AVLTreeNode *X  = W→right;
    W→right = X→left;
    X→left = W;
    W→height = max( Height(W→right), Height(W→left) ) + 1;
    X→height = max( Height(X→right), W→height) + 1;
    return X;
}
```

Time Complexity: O(1). Space Complexity: O(1).

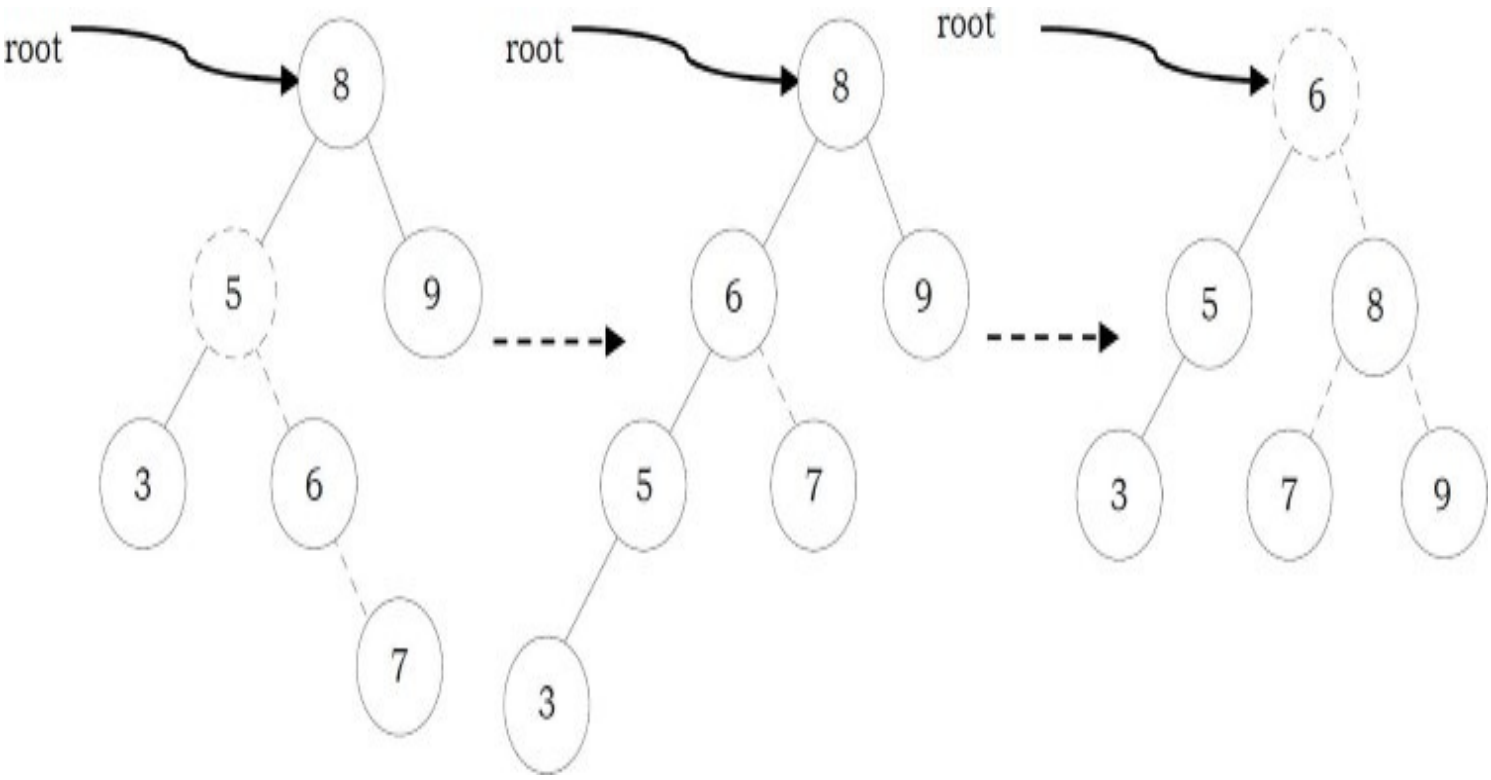


Double Rotations

Left Right Rotation (LR Rotation) [Case-2]: For case-2 and case-3 single rotation does not fix the problem. We need to perform two rotations.



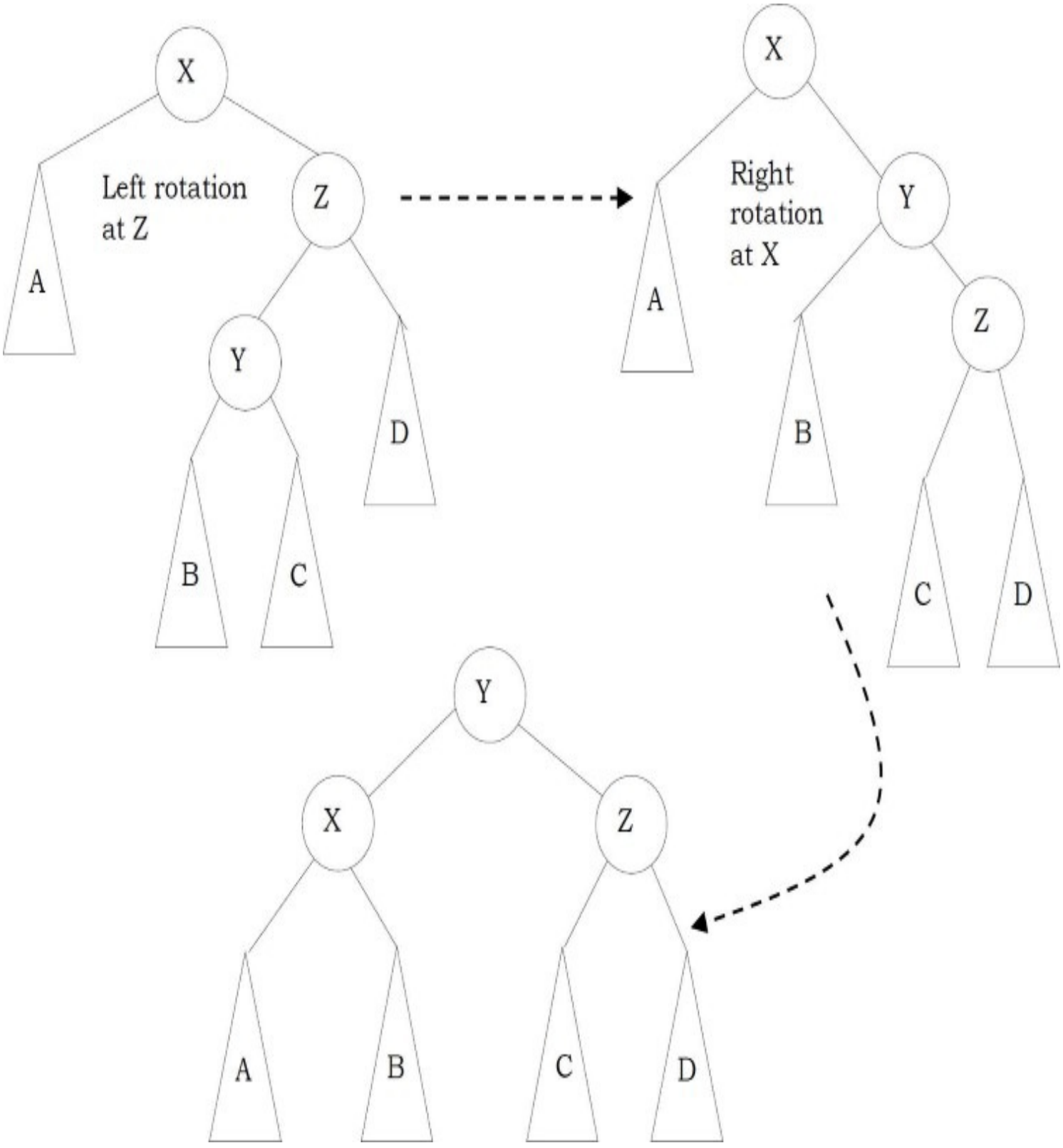
As an example, let us consider the following tree: The insertion of 7 is creating the case-2 scenario and the right side tree is the one after the double rotation.



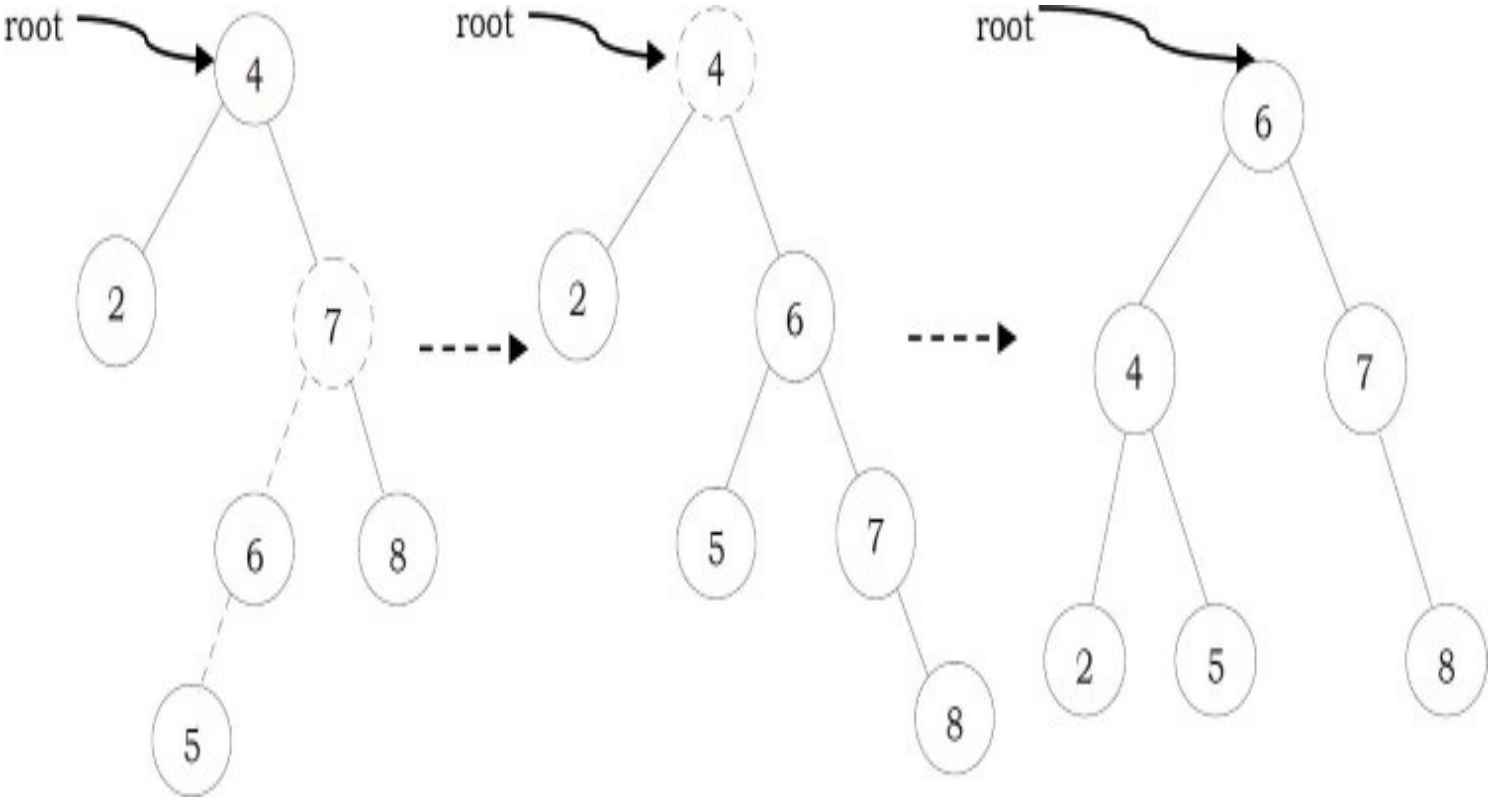
Code for left-right double rotation can be given as:

```
struct AVLTreeNode *DoubleRotatewithLeft( struct AVLTreeNode *Z ){
    Z→left = SingleRotateRight( Z→left );
    return SingleRotateLeft(Z);
}
```

Right Left Rotation (RL Rotation) [Case-3]: Similar to case-2, we need to perform two rotations to fix this scenario.



As an example, let us consider the following tree: The insertion of 6 is creating the case-3 scenario and the right side tree is the one after the double rotation.



Insertion into an AVL tree

Insertion into an AVL tree is similar to a BST insertion. After inserting the element, we just need to check whether there is any height imbalance. If there is an imbalance, call the appropriate rotation functions.

```
struct AVLTreeNode *Insert( struct AVLTreeNode *root, struct AVLTreeNode *parent, int data){
 if( !root) {
        root = (struct AVLTreeNode*) malloc(sizeof (struct AVLTreeNode*) );
        if(!root) {
            printf("Memory Error"); return NULL;
        }
         else {
            root→data = data;
            root→height = 0;
             root→left = root→right = NULL;
         }
 }
 else  if( data < root→data ) {
         root→left = Insert( root→left, root, data );
         if( ( Height( root→left ) - Height( root→right ) ) == 2 ) {
                 if( data < root→left→data )
                         root = SingleRotateLeft( root );
                 else      root = DoubleRotateLeft( root );
         }
 }
 else  if( data > root→data ) {
          root→right = Insert( root→right, root, data );
          if( ( Height( root→right ) - Height( root→left ) ) == 2 ) {
                 if( data < root→right→data )
                     root = SingleRotateRight( root );
                 else  root = DoubleRotateRight( root );
         }
 }
 /* Else data is in the tree already. We'll do nothing */
 root→height = max( Height(root→left), Height(root→right) ) + 1;
 return root;
}
```

Time Complexity: O(*logn*). Space Complexity: O(*logn*).

AVL Trees: Problems & Solutions

Problem-73 Given a height *h*, give an algorithm for generating the *HB*(0).

Solution: As we have discussed, *HB*(0) is nothing but generating full binary tree. In full binary tree the number of nodes with height *h* is: $2^{h+1} - 1$ (let us assume that the height of a tree with one node is 0). As a result the nodes can be numbered as: 1 to $2^{h+1} - 1$.

```
struct BinarySearchTreeNode *BuildHB0(int h){
        struct BinarySearchTreeNode *temp;
        if(h == 0) return NULL;
        temp = (struct BinarySearchTreeNode *) malloc (sizeof(struct BinarySearchTreeNode));
        temp→left = BuildHB0 (h-1);
        temp→data = count++;      //assume count is a global variable
        temp→right = BuildHB0 (h-1);
        return temp;
}
```

Time Complexity: O(*n*).
Space Complexity: O(*logn*), where *logn* indicates the maximum stack size which is equal to height of tree.

Problem-74 Is there any alternative way of solving Problem-73?

Solution: Yes, we can solve it following Mergesort logic. That means, instead of working with height, we can take the range. With this approach we do not need any global counter to be maintained.

```
struct BinarySearchTreeNode *BuildHB0(int l, int r){
    struct BinarySearchTreeNode *temp;
    int mid = l + (r-l)/2;
    if( l > r) return NULL;
    temp = (struct BinarySearchTreeNode *) malloc (sizeof(struct BinarySearchTreeNode));
    temp→data = mid;
    temp→left = BuildHB0(l, mid-1);
    temp→right = BuildHB0(mid+1, r);
    return temp;
}
```

The initial call to the *BuildHBO* function could be: *BuildHB*0(1, 1 ≪ *h*). 1 ≪ *h* does the shift operation for calculating the $2^{h+1} - 1$.

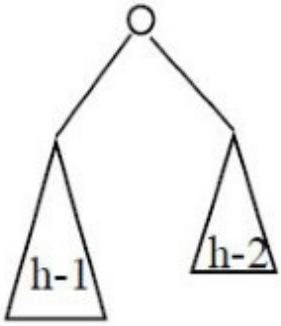
Time Complexity: O(*n*). Space Complexity: O(*login*). Where *logn* indicates maximum stack size which is equal to the height of the tree.

Problem-75 Construct minimal AVL trees of height 0,1,2,3,4, and 5. What is the number of nodes in a minimal AVL tree of height 6?

Solution Let $N(h)$ be the number of nodes in a minimal AVL tree with height h.

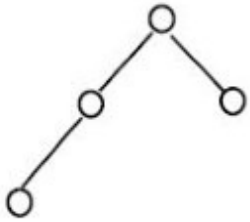
$N(0) = 1$

$N(1) = 2$

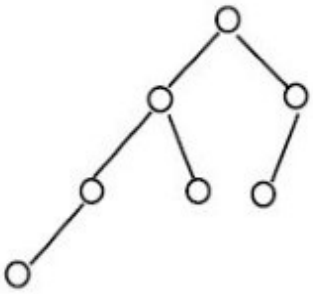


$N(h) = 1 + N(h-1) + N(h-2)$

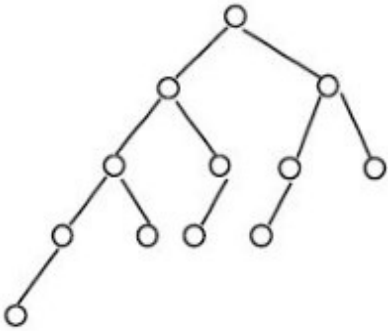
$N(2) = 1 + N(1) + N(0)$
$= 1 + 2 + 1 = 4$



$N(3) = 1 + N(2) + N(1)$
$= 1 + 4 + 2 = 7$



$N(4) = 1 + N(3) + N(2)$
$= 1 + 7 + 4 = 12$



$N(5) = 1 + N(4) + N(3)$
$= 1 + 12 + 7 = 20$

...

Problem-76 For Problem-73, how many different shapes can there be of a minimal AVL tree

of height *h?*

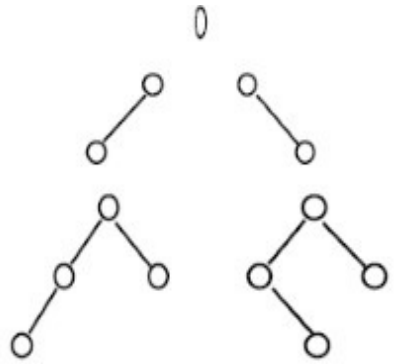
Solution: Let *NS*(*h*) be the number of different shapes of a minimal AVL tree of height *h*.

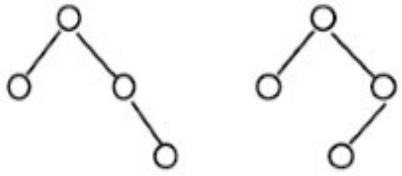
$$NS(0) = 1$$

$$NS(1) = 2$$

$$NS(2) = 2 * NS(1) * NS(0)$$
$$= 2 * 2 * 1 = 4$$

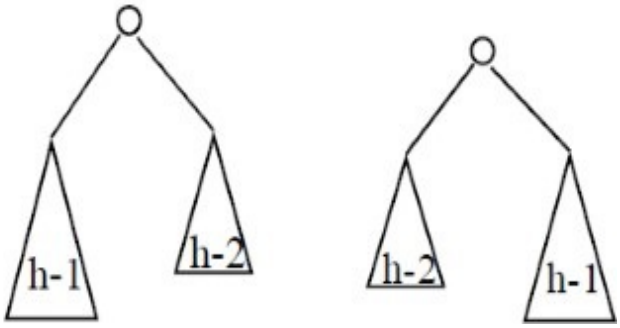
$$NS(3) = 2 * NS(2) * NS(1)$$
$$= 2 * 4 * 1 = 8$$





...

$$NS(h) = 2 * NS(h-1) * NS(h-2)$$



Problem-77 Given a binary search tree, check whether it is an AVL tree or not?

Solution: Let us assume that *IsAVL* is the function which checks whether the given binary search tree is an AVL tree or not. *IsAVL* returns –1 if the tree is not an AVL tree. During the checks each node sends its height to its parent.

```
int IsAVL(struct BinarySearchTreeNode *root){
        int left, right;
        if(!root)  return 0;
        left = IsAVL(root→left);
        if(left == -1)
            return left;
        right = IsAVL(root→right);
        if(right == -1)
             return right;
        if(abs(left-right)>1)
             return -1;
        return Max(left, right)+1;
}
```

Time Complexity: O(*n*). Space Complexity: O(*n*).

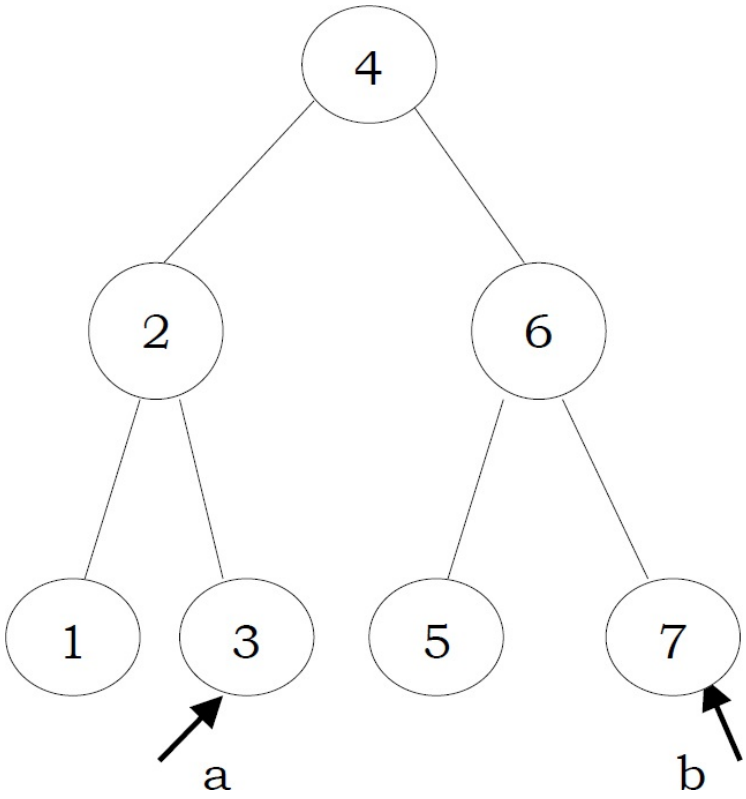
Problem-78 Given a height *h*, give an algorithm to generate an AVL tree with minimum number of nodes.

Solution: To get minimum number of nodes, fill one level with $h - 1$ and the other with $h - 2$.

```
struct AVLTreeNode *GenerateAVLTree(int h){
        struct AVLTreeNode *temp;
        if(h == 0) return NULL;
        temp = (struct AVLTreeNode *)malloc (sizeof(struct AVLTreeNode));
        temp→left = GenerateAVLTree(h-1);
        temp→data = count++; //assume count is a global variable
        temp→right = GenerateAVLTree(h-2);
        temp→height = temp→left→height+1; // or temp→height = h;
        return temp;
}
```

Problem-79 Given an AVL tree with *n* integer items and two integers *a* and *b*, where *a* and *b* can be any integers with $a <= b$. Implement an algorithm to count the number of nodes in the range [*a*,*b*].

Solution:



The idea is to make use of the recursive property of binary search trees. There are three cases to consider: whether the current node is in the range [*a, b*], on the left side of the range [*a, b*], or on the right side of the range [*a,b*]. Only subtrees that possibly contain the nodes will be processed under each of the three cases.

```
int RangeCount(struct AVLNode *root, int a, int b) {
        if(root == NULL) return 0;
        else if(root→data > b)
                return RangeCount(root→left, a, b);
        else if(root→data < a)
                return RangeCount(root→right, a, b);
        else if(root→data >= a && root→data <= b)
                return RangeCount(root→left, a, b) + RangeCount(root→right, a, b) + 1;
}
```

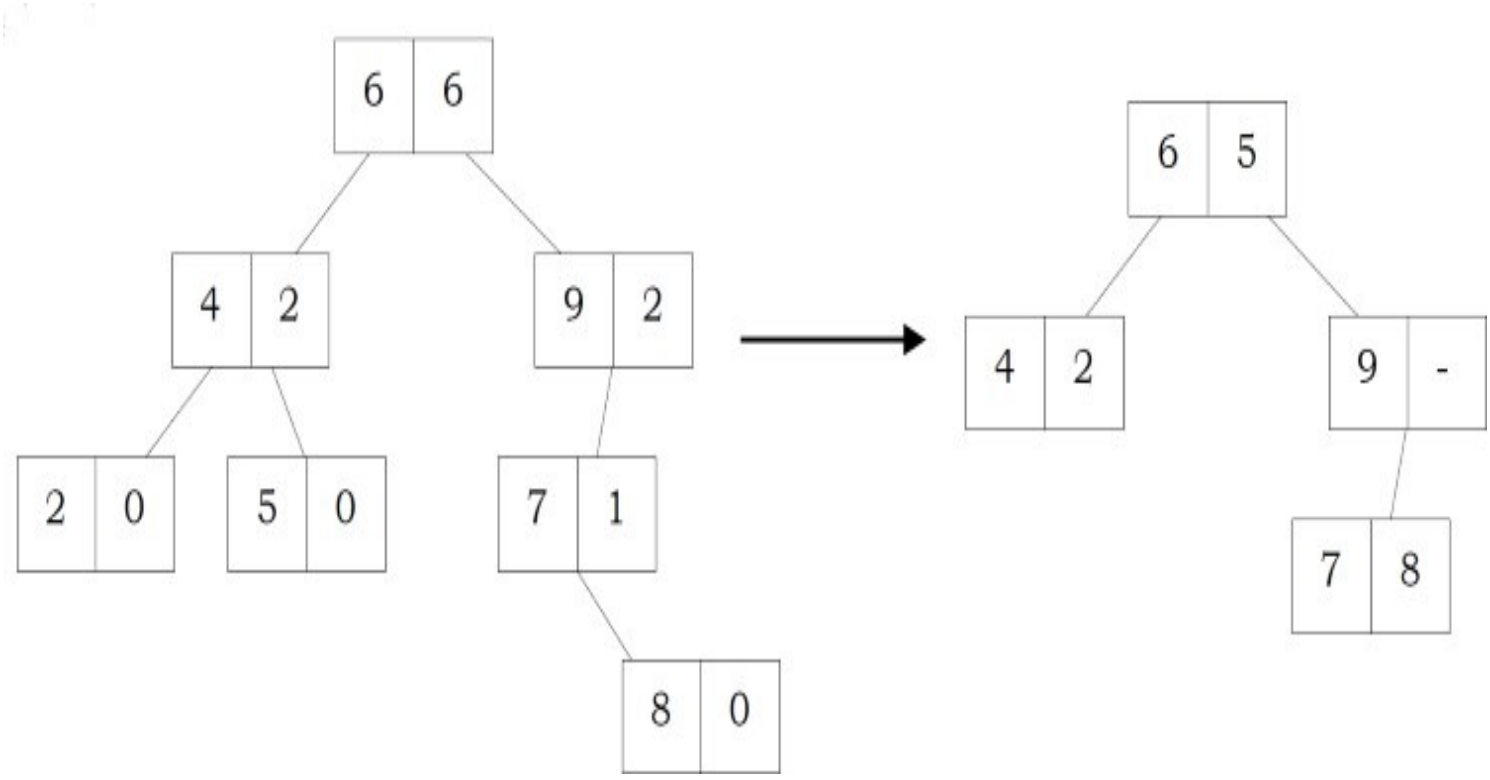
The complexity is similar to *in – order* traversal of the tree but skipping left or right sub-trees when they do not contain any answers. So in the worst case, if the range covers all the nodes in the tree, we need to traverse all the *n* nodes to get the answer. The worst time complexity is therefore O(*n*).

If the range is small, which only covers a few elements in a small subtree at the bottom of the tree, the time complexity will be O(*h*) = O(*logn*), where *h* is the height of the tree. This is because only a single path is traversed to reach the small subtree at the bottom and many higher level subtrees

have been pruned along the way.

Note: Refer similar problem in BST.

Problem-80 Given a BST (applicable to AVL trees as well) where each node contains two data elements (its data and also the number of nodes in its subtrees) as shown below. Convert the tree to another BST by replacing the second data element (*n*umber of nodes in its subtrees) with previous node data in inorder traversal. Note that each node is merged with *inorder* previous node data. Also make sure that conversion happens in-place.



Solution: The simplest way is to use level order traversal. If the number of elements in the left subtree is greater than the number of elements in the right subtree, find the maximum element in the left subtree and replace the current node second data element with it. Similarly, if the number of elements in the left subtree is less than the number of elements in the right subtree, find the minimum element in the right subtree and replace the current node *second* data element with it.

```
struct BST *TreeCompression (struct BST *root){
        struct BST *temp, *temp2;
        struct Queue *Q = CreateQueue();
        if(!root) return;
        EnQueue(Q, root);
        while(!IsEmptyQueue(Q)) {
                temp = DeQueue(Q);
                if(temp→left && temp→right && temp→left→data2 > temp→right→data2)
                    temp2 = FindMax(temp);
                else temp2 = FindMin(temp);
                 temp→data2 = temp2→data2;   //Process current node
               //Remember to delete this node.
                DeleteNodeInBST(temp2);
                  if(temp→left)
                            EnQueue(Q, temp→left);
                  if(temp→right)
                            EnQueue(Q, temp→right);
        }
        DeleteQueue(Q);
}
```

Time Complexity: O(*nlogn*) on average since BST takes O(*logn*) on average to find the maximum or minimum element. Space Complexity: O(*n*). Since, in the worst case, all the nodes on the entire last level could be in the queue simultaneously.

Problem-81 Can we reduce time complexity for the previous problem?

Solution: Let us try using an approach that is similar to what we followed in Problem-60. The idea behind this solution is that inorder traversal of BST produces sorted lists. While traversing the BST in inorder, keep track of the elements visited and merge them.

```
struct BinarySearchTreeNode * TreeCompression(struct BinarySearchTreeNode *root,
                                              int *previousNodeData){
    if(!root) return NULL;
        TreeCompression(root→left, previousNode);
    if(*previousNodeData == INT_MIN){
        *previousNodeData = root→data;
        free(root);
    }
    if(*previousNodeData != INT_MIN){                    //Process current node
        root→data2 = previousNodeData;
        *previousNodeData = INT_MIN;
    }
    return TreeCompression(root→right, previousNode);
}
```

Time Complexity: O(*n*).

Space Complexity: O(1). Note that, we are still having recursive stack space for inorder traversal.

Problem-82 Given a BST and a key, find the element in the BST which is closest to the given key.

Solution: As a simple solution, we can use level-order traversal and for every element compute the difference between the given key and the element's value. If that difference is less than the previous maintained difference, then update the difference with this new minimum value. With this approach, at the end of the traversal we will get the element which is closest to the given key.